

Table of Contents

Table of Contents	1
1. Introduction	2
1.1 Purpose.....	2
1.2 Document Convention	2
1.3 Intended Audience and Reading Sugestions	2
1.4 Project Scope	3
1.5 References	3
2. Overall Description	4
2.1 Product Perspective.....	4
2.2 Product Features	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraint	7
2.6 User Documentation.....	8
2.7 Assumption and Dependencies.....	8
3. System Features.....	9
3.1 Sponsor Features	9
3.2 Influencers Features.....	10
3.3 Admin Feature.....	11
4. External Requirements	12
4.1 User Interfaces.....	12
4.2 Hardware Interfaces	13
4.3 Software Interfaces	14
4.4 Communications Interfaces.....	14
5. Feasibility Study	15
5.1 Technical Feasibility.....	15
5.2 Operational Feasibility	15
5.3 Economic Feasibility	15
5.4 Schedule Feasibility.....	15
6. Development Methodology	16
6.1 Iterative Waterfall model.....	16
7. Nonfunctional Requirements	17
7.1 Performance Requirements.....	17
7.2 Safety Requirements	18
8. Model Diagrams	18
8.1 Use Case Diagram	18
8.2 Class Diagram.....	20
8.3 Data Flow Diagram	23
8.4 ER Diagram	25
8.5 Activity Diagram	28
9. Codes	33
9.1 Code File Structure	33
9.2 Codes	33
10. Screenshots.....	89
APPENDIX : ISSUES LIST.....	93

1. Introduction

1.1 Purpose

The purpose of SponsorSphere is to provide a structured platform for sponsors and influencers to collaborate efficiently on advertising campaigns. By simplifying the processes of campaign management, ad-request handling, and user interactions, SponsorSphere bridges the gap between sponsors seeking influencer marketing and influencers seeking monetization. Sponsors can create and manage campaigns, send ad requests, and monitor results, while influencers can view and respond to these requests, negotiate terms, and promote products or services to their followers, streamlining sponsor-influencer coordination by offering role-based dashboards and search functionalities. SponsorSphere is designed to facilitate efficient campaign management and brand promotion, benefiting both parties through a structured and user-friendly environment.

1.2 Document Convention

This document uses the following conventions:

- **Bold:** Indicates key terms or concepts that are important within the context of the document.
- *Italic:* Used for emphasis or to denote specific titles, names of documents, applications, or sections.
- **List Items:** Bullet points are used to enumerate features, requirements, or important notes for clarity and easy reference.
- **Headings:** Major sections are marked with headings of different levels to structure the document and aid in navigation.
- **References:** URLs and external resources are provided in a consistent format for easy access and further reading.

1.3 Intended Audience and Reading Sugestions

This document is intended for:

- **Project Stakeholders:** Individuals involved in the project's development and maintenance.
- **End Users:** Sponsors, influencers, and administrators who will use the platform.
- Reading Suggestions:
 - Stakeholders should focus on sections detailing the project scope and system overview.

- End users might find the user interface descriptions and functionalities most relevant.

1.4 Project Scope

SponsorSphere is a platform designed to support the collaboration between sponsors and influencers through campaign management, ad requests, and profile functionalities, with a focus on efficient user role management.

Objectives:

- **Facilitate Campaign Coordination:** Streamline the process of connecting sponsors with relevant influencers.
- **Enhance Sponsor and Influencer Efficiency:** Reduce manual tasks by automating ad requests, responses, and campaign tracking.
- **Provide Insights:** Offer real-time analytics on campaign performance, ad reach, and user engagement for informed decision-making.

Benefits:

- **Time Savings:** Automation speeds up campaign management for sponsors and response handling for influencers.
- **Improved Transparency:** Sponsors and influencers can track ad request status, budgets, and campaign details easily.
- **Data-Driven Decisions:** Simplifies data collection and analysis for better campaign targeting and influencer selection.

SponsorSphere supports the goals of digital marketing and brand engagement by integrating technology into influencer marketing, aligning with industry best practices for brand promotion.

1.5 References

1. Flask Documentation: <https://flask.palletsprojects.com>
2. Jinja2 Documentation: <https://jinja.palletsprojects.com>
3. SQLite Documentation: <https://www.sqlite.org/docs.html>

4. IEEE 830-1998: IEEE Recommended Practice for Software Requirements Specifications.

2. Overall Description

2.1 Product Perspective

SponsorSphere is a web application specifically designed to facilitate interactions between sponsors and influencers, enabling seamless campaign management, ad request handling, and data-driven collaboration. Built on a stack that includes Flask for backend logic, Jinja2 templates with Bootstrap for responsive and user-friendly UI, and SQLite for lightweight data management, SponsorSphere is optimized for local deployment and easy usage.

Context and Origin:

- **Motivation:** With the growth of social media and the increasing value of influencer marketing, brands need a reliable way to connect with influencers who can promote their products effectively. SponsorSphere addresses this need by creating a centralized platform for sponsors to find, negotiate, and collaborate with influencers, all while simplifying campaign tracking and data analysis.
- **Target Audience:** Ideal for brands, individual sponsors, and influencers, SponsorSphere streamlines campaign management, ad requests, and engagement tracking.

Relation to Larger Systems:

SponsorSphere can operate independently as a stand-alone platform but has the potential to integrate with other marketing and analytics tools, making it a versatile addition to a company's digital marketing stack. In future iterations, SponsorSphere may connect to:

- **Social Media Platforms:** Direct integration with social media APIs (e.g., Instagram, Twitter) could provide real-time data on influencer reach, engagement, and audience demographics.
- **Analytics Tools:** Integration with external analytics tools (such as Google Analytics or Chart.js) for enhanced campaign performance insights and visualization, allowing sponsors to make data-driven decisions.

2.2 Product Features

SponsorSphere provides several key features that enhance lab management by integrating with GitHub to automate tracking of student assignments. The major features are organized as follows:

1. User Management

- **Role-Based Registration:** Enables sponsors, influencers, and admins to sign up and create profiles within the system.
- **Profile Management:** Allows influencers to update their profiles with details like niche, reach, and category, which are publicly visible to sponsors..

2. Campaign Management

- **Campaign Creation:** Sponsors can create, update, and categorize campaigns with goals, budgets, start/end dates, and visibility settings.
- **Public/Private Campaigns:** Sponsors can set campaigns as public for all influencers or private for selected ones only.

3. Ad Request Management

- **Ad Request Creation:** Sponsors can send ad requests to influencers with specific requirements, such as payment amount, deliverables, and timelines.
- **Negotiation and Tracking:** Influencers can accept, reject, or negotiate ad requests, allowing for flexible terms. Statuses are tracked as pending, accepted, or rejected.

4. Search and Discovery

- **Influencer Search:** Sponsors can search for influencers based on niche, reach, and other relevant metrics.
- **Campaign Search:** Influencers can search for active, public campaigns that align with their audience and interests.

5. Reporting and Analytics

- **Campaign Performance Reports:** Provides real-time analytics on campaign reach, engagement, and ROI for sponsors.
- **User Activity Tracking:** Admins can monitor ad request statuses, flagged users, and campaign statistics to ensure transparency and security.

2.3 User Classes and Characteristics

1. Sponsors

- *Usage:* Frequent (for campaign creation and influencer engagement).
- *Functions:* Create and manage campaigns, send ad requests, search for influencers, view campaign analytics.
- *Skills:* Basic web platform navigation and understanding of influencer marketing..
- *Privileges:* Access to manage their own campaigns and view selected influencer profiles.
- *Importance:* High (primary revenue-generating users).

2. Influencers

- *Usage:* Regular (for responding to ad requests and searching for campaigns).
- *Functions:* View and accept/reject ad requests, negotiate terms, update public profile, search for campaigns.
- *Skills:* Basic platform usage and knowledge of campaign engagement strategies.
- *Privileges:* Access to their profile and public campaigns; limited interaction with sponsors through ad requests.
- *Importance:* High (primary service providers for sponsors' campaigns).

3. Administrators

- *Usage:* Infrequent (for system monitoring and user management).
- *Functions:* Oversee user activities, manage flagged users or campaigns, view overall platform statistics.
- *Skills:* High technical expertise (Flask, databases, basic system troubleshooting).
- *Privileges:* Full access to monitor and maintain all user data, campaigns, and ad requests.
- *Importance:* Moderate (support role, ensuring platform integrity and compliance).

2.4 Operating Environment

1. Hardware Platform:

- Runs on standard web servers (cloud-based or on-premises).
- Accessible via desktops, laptops, or mobile devices for end-users.

2. Operating System:

- *Server*: Linux (Ubuntu, Debian) or Windows Server.
- *Client*: Any OS (Windows, macOS, Linux, Android, iOS) with browser access.

3. Software Components:

- *Backend*: Flask (Python framework) for application logic.
- *Database*: SQLite for lightweight data storage, suitable for handling the application's needs.
- *Frontend*: HTML, CSS, JavaScript, along with Bootstrap for responsive design.

4. Dependencies:

- Python 3.x and Django libraries.

5. Browser Compatibility and Network Requirements:

- Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- Reliable internet connection and web access.

SponsorSphere is designed to operate effectively in diverse environments, requiring only basic network access and a web browser to ensure a smooth user experience.

2.5 Design and Implementation Constraint

1. Technologies and Tools :

- Must use Flask as the backend framework, with Jinja2 for templating and Bootstrap for frontend design.
- SQLite is required as the primary database for local data management

2. Security Requirements :

- Implementation of user authentication for Admins, Sponsors, and Influencers to ensure secure access to the platform.
- Utilize secure password hashing and storage methods to protect user credentials.

3. Maintenance Responsibility : The application must follow Flask conventions and best practices to facilitate ease of maintenance and updates by future developers.

4. Resource Constraints : Efficient management of SQLite database operations to prevent performance degradation, especially during simultaneous user interactions in a local deployment environment.

2.6 User Documentation

1. User Manual:

- Comprehensive instructions for Admins, Sponsors, and Influencers on how to navigate and utilize the platform.
- Contains detailed walkthroughs, including screenshots and step-by-step guides for key functionalities such as campaign management, ad request handling, and user profile management.
- *Format:* PDF

2. Quick Start Guide:

- A concise overview of essential tasks (e.g., logging in, creating campaigns, sending ad requests, and managing profiles).
- Provides tips for effective use of the platform and common troubleshooting advice.
- *Format:* PDF

2.7 Assumption and Dependencies

1. Assumptions :

- Users (Admins, Sponsors, and Influencers) have reliable internet access for seamless interaction with the platform.
- The Flask framework and SQLite database are stable and adequately support the required functionalities.
- Users possess basic familiarity with web applications and online marketing concepts.
- All user roles can effectively navigate the web interface.
- The development team has the necessary resources and support for local deployment and testing.

2. Dependencies :

- **Flask Framework:** Required for backend development and handling user interactions.
- **SQLite Database:** Essential for data storage and management of user and campaign information.
- **Bootstrap:** Utilized for responsive frontend design to ensure a user-friendly interface across devices.

3. System Features

3.1 Sponsor Features

3.1.1 Campaign Management

Description : This feature allows sponsors to create, manage, and track their advertising campaigns effectively.

Priority : High

Stimulus/Response Sequences :

- *Stimulus :* Sponsor creates a new campaign.
Response : System saves the campaign details and adds it to the sponsor's dashboard.
- *Stimulus :* Sponsor updates an existing campaign.
Response : System updates the campaign details and reflects the changes on the dashboard.

Functional Requirements :

- *REQ-1 :* Allow sponsors to create new campaigns with fields such as name, description, start date, end date, budget, and visibility.
- *REQ-2 :* Enable sponsors to update existing campaign details.
- *REQ-3:* Allow sponsors to delete campaigns as needed

3.1.2 Ad Request Management

Description : Enables sponsors to send ad requests to influencers and manage responses.

Priority : High

Stimulus/Response Sequences :

- *Stimulus :* Sponsor sends an ad request to an influencer.

Response : System logs the request and updates the status.

- *Stimulus* : Sponsor modifies an existing ad request.

Response : System updates the request details accordingly.

Functional Requirements :

- *REQ-1* : Allow sponsors to create ad requests specifying goals, influencer ID, requirements, payment amount, and status.
- *REQ-2* : Enable sponsors to edit or delete existing ad requests.

3.1.3 Search Functionality

Description : Sponsors can search for influencers based on various criteria, such as niche and reach.

Priority : High

Stimulus/Response Sequences :

- *Stimulus* : Sponsor initiates a search for influencers.

Response : System displays a list of matching influencers based on the search criteria.

Functional Requirements :

- *REQ-1* : Provide search options for sponsors to filter influencers by niche, reach, and budget.

3.2 Influencers Features

3.2.1 Ad Request Management

Description : Influencers can view and respond to ad requests from sponsors.

Priority : High

Stimulus/Response Sequences :

- *Stimulus* : Influencer receives an ad request.

Response : System displays the request details on the influencer's dashboard.

- *Stimulus* : Influencer accepts or rejects the ad request.

Response : System updates the status of the request accordingly.

Functional Requirements :

- *REQ-1* : Allow influencers to view ad requests with details such as goals and payment terms.
- *REQ-2* : Enable influencers to accept, reject, or negotiate payment amounts on ad requests.

3.2.2 Profile Management

Description : Influencers can manage their public profiles, showcasing their reach, niche, and engagement metrics.

Priority : Medium

Stimulus/Response Sequences :

- *Stimulus* : Influencer updates their profile information.

Response : System saves the updated profile and reflects changes publicly.

Functional Requirements :

- *REQ-1* : Allow influencers to edit their public profiles, including categories and reach.
- *REQ-2* : Display updated profiles on the platform for sponsors to view.

3.3 Admin Feature**3.3.1 User Management**

Description : Admins have the ability to oversee user activities, manage roles, and monitor flagged content..

Priority : High

Stimulus/Response Sequences :

- *Stimulus* : Admin reviews user statistics.

Response : System displays current statistics on active users and flagged content.

- *Stimulus* : Admin flags a user or campaign.

Response : System logs the action and updates the user status.

Functional Requirements :

- *REQ-1* : Allow admins to monitor and manage users, campaigns, and their statuses.
- *REQ-2* : Enable admins to flag inappropriate content for review.

3.3.2 Analytics Dashboard

Description : Admins can access an analytics dashboard to track overall platform performance and user engagement.

Priority : Medium

Stimulus/Response Sequences :

- *Stimulus* : Admin accesses the analytics dashboard.

Response : System generates a report on user engagement and campaign performance.

Functional Requirements :

- *REQ-1* : Provide real-time analytics on user activities and campaign performance metrics..
- *REQ-2* : Allow admins to export analytics reports for external review.

4. External Requirements

4.1 User Interfaces

SponsorSphere's user interface is designed to be intuitive and engaging, featuring a modern aesthetic that aligns with the branding of the platform. It includes essential sections for both sponsors and influencers, emphasizing straightforward navigation and clear functionalities..

Key Interface Elements:

- **Login Screens:**
 - Separate login pages for sponsors and influencers with user-friendly input fields.
 - Error messages displayed in red below the input fields for clarity.
- **Sponsors Dashboard:**
 - Displays current campaigns, ad requests, and influencer responses in a card-based layout.
 - Buttons for creating new campaigns, managing requests, and accessing analytics reports.

- **Influencers Dashboard:**

- Sections for viewing ad requests, profile information, and campaign engagement.
- Tabs for navigating past collaborations and tracking performance metrics.

- **Campaign Management:**

- Sponsors can add, edit, and view campaign details with dropdowns for categories and targeted influencers.
- Inline error messages appear in red for any validation issues.

General Interface Characteristics:

- **Color Scheme:** Consistent branding colors used for buttons, backgrounds, and highlights.
- **Navigation:** Fixed top navigation bar for main actions, with standard buttons for submissions and edits.
- **Error Messaging:** Inline error alerts displayed in red to enhance user experience.

4.2 Hardware Interfaces

SponsorSphere is a web-based application that requires minimal hardware-specific interactions. The system supports standard computing hardware with the following characteristics:

Supported Devices:

- **Desktop Computers:** Running modern browsers like Chrome, Firefox, Edge.
- **Laptops:** Compatible with standard web browsers.
- **Tablets and Mobile Devices:** Optimized for smaller screens through responsive web design.

Data and Control Interactions:

- **Client-Server Interaction:** User devices (client-side) communicate with the server over HTTPS, ensuring secure data transfer.
- **Server:** Hosted on a machine with sufficient capacity to handle database operations, API interactions, and user requests.

SponsorSphere does not require specialized hardware; any standard internet-enabled device with a browser is sufficient for access.

4.3 Software Interfaces

SponsorSphere integrates with several software components to ensure smooth functionality:

Operating System:

- **Server:** Runs on Linux-based OS (e.g., Ubuntu) for hosting the Flask application.
- **Client:** Compatible with any OS that supports modern web browsers.

Web Framework:

- **Flask:** Manages server-side operations, authentication, and database interactions.

Database:

- **SQLite:** Stores user data, campaigns, influencer profiles, and transaction logs.
 - **Data In:** User details and campaign inputs.
 - **Data Out:** Reports, analytics, and user engagement metrics.

APIs:

- **Communication:** Utilizes HTTPS with REST API calls for secure data exchange.

Libraries:

- **SQLAlchemy:** Facilitates database interactions and ORM functionalities.
- **Requests:** For making API calls to external services.

Secure communications are ensured via HTTPS, with data consistency maintained through application's ORM.

4.4 Communications Interfaces

SponsorSphere requires the following communication protocols and standards:

- **Web Browser Access:** Communication between client and server happens over HTTPS, ensuring secure transmission of data.
- **API Integration:** Data is fetched via secure HTTPS REST API calls, using standard JSON formatting for data exchange.

- **Server Communication:** Internal communication between the web server and database uses standard TCP/IP protocols.

All data exchanges are encrypted using SSL/TLS to maintain security and privacy.

5. Feasibility Study

5.1 Technical Feasibility

SponsorSphere is built on widely accessible technologies such as Flask for backend development, SQLite for data management, and Jinja2 for templating, with Bootstrap for responsive design. These tools are compatible with standard desktop and mobile operating systems, allowing efficient local deployment. The system's required technologies, including Python libraries and web frameworks, are well-documented and have strong community support, reducing the learning curve for developers. Additionally, hardware requirements are minimal as it operates on standard web servers, making it technically feasible within the existing infrastructure.

5.2 Operational Feasibility

SponsorSphere offers a role-based user experience tailored to sponsors, influencers, and administrators. Its user-friendly interfaces simplify campaign management, ad requests, and data tracking, minimizing the need for extensive training. Administrators can oversee platform activities effectively, while sponsors and influencers can interact intuitively through personalized dashboards. The project is operationally feasible, given its alignment with user requirements and the current technological capabilities of the target audience.

5.3 Economic Feasibility

A cost-benefit analysis indicates that the potential benefits, such as enhanced campaign efficiency, time savings, and increased engagement, outweigh the initial and maintenance costs. The investment in SponsorSphere provides sponsors with streamlined marketing processes, potentially leading to increased revenue and improved influencer relationships. Therefore, the project is economically viable, with significant long-term gains expected from enhanced operational efficiency and user satisfaction.

5.4 Schedule Feasibility

SponsorSphere's development follows the Agile Model, enabling iterative progress and continuous feedback. The technology stack is efficient and familiar to the development team,

enabling a streamlined workflow. Considering the project scope and the availability of experienced developers, SponsorSphere is expected to be completed within a six-month timeframe. The phased release approach aligns with realistic project milestones, ensuring a feasible and timely launch.

In summary, SponsorSphere is a feasible project from a technical, operational, economic, and schedule perspective. The planned approach addresses potential risks while capitalizing on SponsorSphere's anticipated advantages.

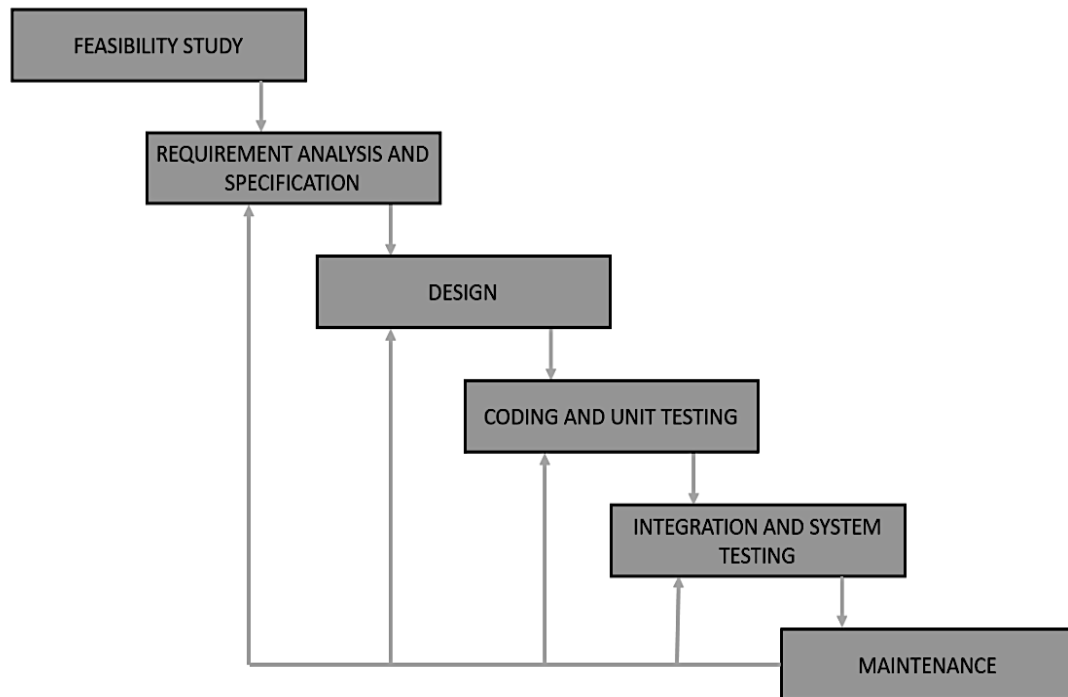
6. Development Methodology

6.1 Iterative Waterfall model

For the development of SponsorSphere, the **Iterative Waterfall model** is adopted. This methodology combines the structured phases of the traditional Waterfall model with the flexibility of iterative design, allowing enhancements at each phase before moving to the next. Unlike the strict sequential approach, this model provides feedback loops from each phase to its preceding phase, enabling early error correction and adaptation to evolving requirements. The stages of development are as follows:

1. **Requirements Gathering:** Business stakeholders and developers collaborate to identify and document the project's goals and requirements. This stage establishes a clear understanding of what the system aims to achieve, ensuring that initial expectations align with the business objectives.
2. **Design:** Based on the gathered requirements, developers create a preliminary design of SponsorSphere. This stage includes architectural planning and UI/UX design, resulting in blueprints that guide the implementation phase.
3. **Implementation:** Developers build the application according to the design specifications. Coding, database configuration, and API integrations occur during this stage, with an emphasis on creating a functional system aligned with the initial design.
4. **Testing:** The application undergoes rigorous testing to verify that all features function as intended. Any identified issues are addressed, ensuring that the product meets the documented requirements and performs reliably.

5. **Deployment:** SponsorSphere is then deployed to a live environment, making it accessible to end users. This stage includes configuring the hosting environment and ensuring that deployment processes run smoothly.
6. **Review and Improvement:** After deployment, the system's performance is monitored, and feedback from end users is gathered. Developers and stakeholders collaboratively review these insights to identify areas for improvement, initiating the next iteration.



This iterative process continues until SponsorSphere fully meets the project's objectives and end-user expectations. Each iteration builds upon previous ones, facilitating a continuous improvement cycle that enhances the product's functionality, reliability, and usability.

7. Nonfunctional Requirements

7.1 Performance Requirements

SponsorSphere must meet the following performance criteria:

- **Response Time:** Pages should load within 2 seconds for standard requests (e.g., dashboard view, campaign listings). Data-intensive operations, such as analytics report generation, should complete within 5 seconds..

- **Data Update:** Data updates from influencer submissions and campaign analytics should not exceed 10 minutes to complete.
- **Simultaneous Users:** The system should support up to 100 concurrent users without noticeable performance degradation.
- **Database Queries:** Should be optimized to handle complex queries with minimal latency.
- **Scalability:** The system should maintain performance standards as the number of users and data volume increases, allowing for easy scaling of server resources.

Performance goals ensure a smooth user experience for both sponsors and influencers, even with increasing platform usage.

7.2 Safety Requirements

SponsorSphere incorporates the following safety measures:

- **Data Integrity:** The system should uphold data integrity for influencer and sponsor data, enabling easy retrieval and updates whenever needed.
- **Access Control:** Role-based access limits features based on user roles (e.g., sponsor, influencer) to prevent unauthorized actions.
- **Error Handling:** Robust error handling mechanisms prevent data corruption and unauthorized access during unexpected situations or failures.
- **Regulations Compliance:** System design adheres to relevant data privacy and security policies, ensuring that sensitive user information is protected.

These safeguards are in place to prevent data loss and unauthorized access while ensuring reliable and secure system operations.

8. Model Diagrams

8.1 Use Case Diagram

A use case diagram in the UML is a type of behavioural diagram. A use case diagram is used to represent the dynamic behaviour of a system. It is a visual representation of how a system interacts with its external actors (which can be users or other systems) and the various actions or use cases that the system performs in response to those interactions.

Key elements of a use case diagram include:

- **Actors:** Represents an external entity that interacts with the system. Actors can be individuals, other systems, or even hardware components.



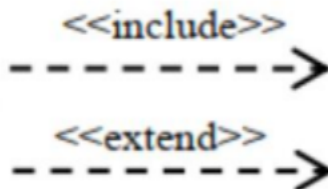
- **Use Cases:** Use cases represent specific functionalities or actions that the system can perform. Each use case is a discrete piece of functionality that provides value to one or more actors. Use cases are represented as ovals in the diagram.



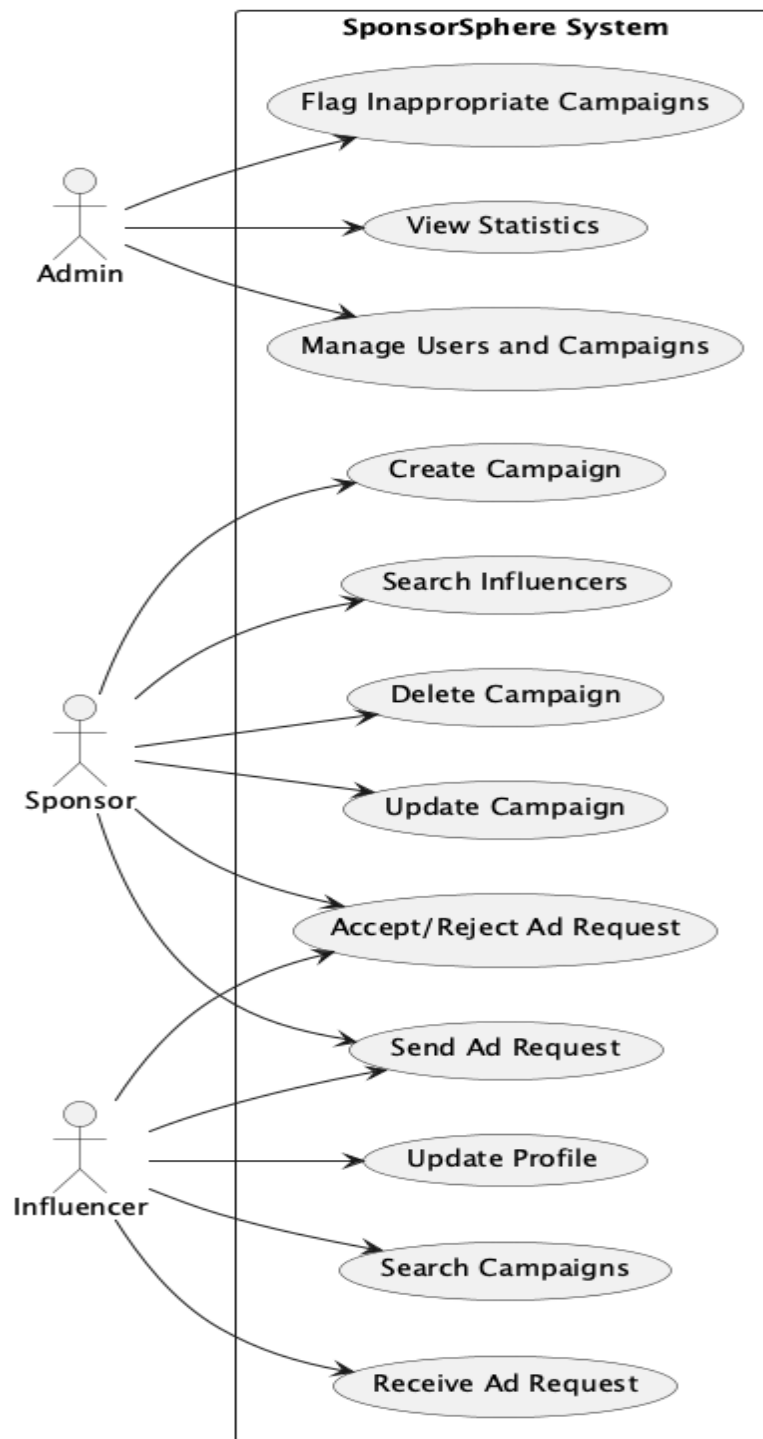
- **Associations:** Lines (also known as associations) connect actors to use cases. These lines show which actors are involved in each use case.



- **Include and Extend Relationships:** Sometimes, use cases have relationships with other use cases. "Include" relationships indicate that one use case includes another as part of its behavior. "Extend" relationships indicate that one use case can extend the behavior of another under certain conditions.



A. Use Case Diagram for SponsorSphere System



8.2 Class Diagram

Class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

Class diagrams are used to model the static structure of a system, meaning that they show the different types of objects that exist in the system and how they are related to each other. Class

diagrams can be used to model any type of system, from simple websites to complex enterprise applications.

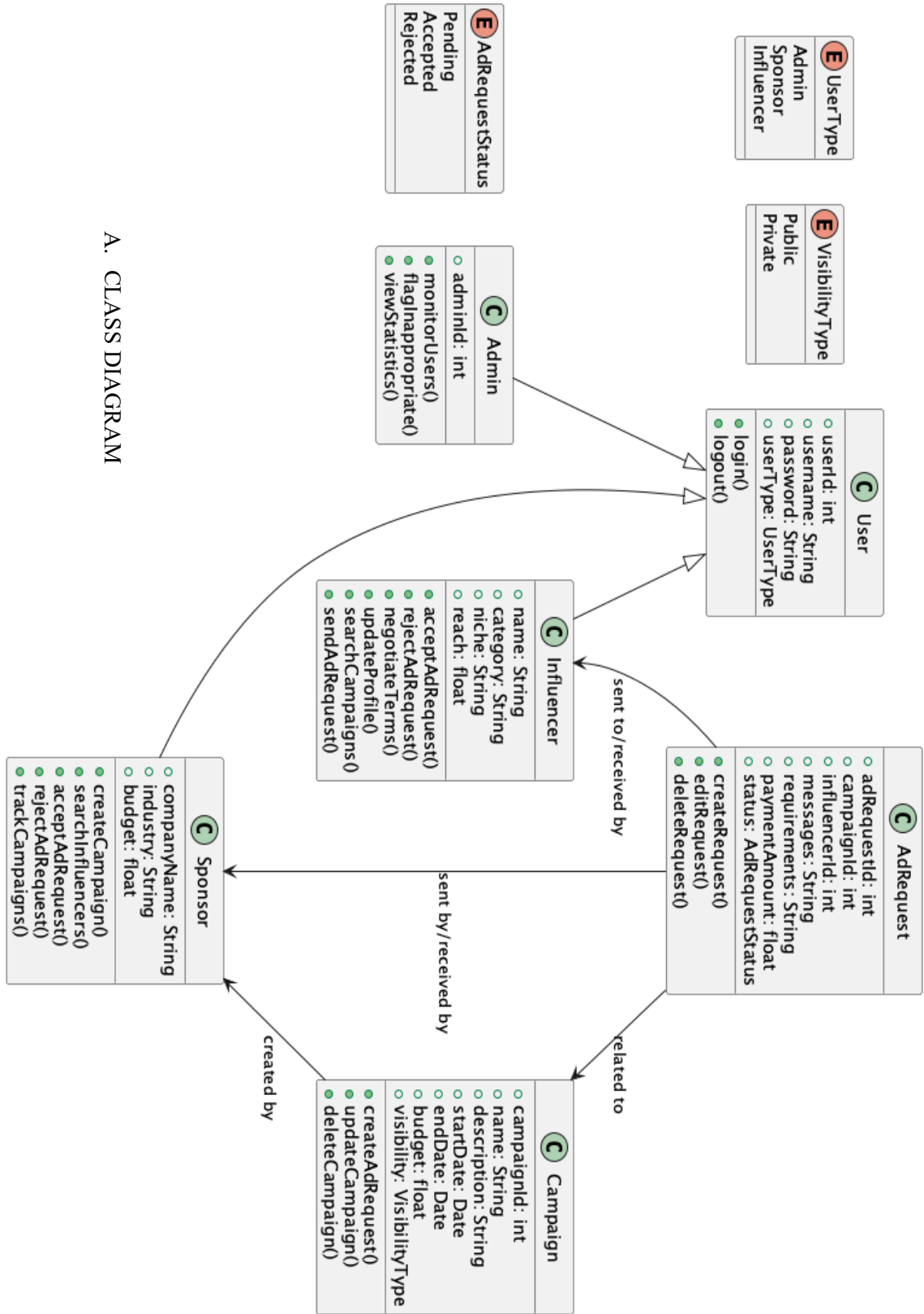
Class diagrams are made up of the following elements:

- **Classes:** Classes are the basic building blocks of class diagrams. A class represents a type of object, such as a customer, a product, or an order.
- **Attributes:** Attributes represent the characteristics of a class. For example, a customer class might have attributes such as name, address, and phone number.
- **Operations:** Operations represent the actions that can be performed on a class.
- **Relationships:** Relationships show how different classes are related to each other.

There are four main types of relationships in class diagrams:

- **Association:** An association shows that two classes are related to each other in some way.
- **Aggregation:** An aggregation shows that one class is a part of another class.
- **Composition:** A composition shows that one class cannot exist without the other class.
- **Generalization:** A generalization shows that one class is a subclass of another class.





A. CLASS DIAGRAM

8.3 Data Flow Diagram

A Data Flow Diagram (DFD) represents the flow of data within a system or process, providing insight into the inputs, outputs, and processes involved. Unlike flowcharts, DFDs do not include control flow, loops, or decision rules. They belong to structured-analysis modeling tools and are popular for visualizing major steps and data involved in software-system processes.

Components of DFD

The Data Flow Diagram has 4 components:

- **Process:** Transforms input into output within a system. Represented by rectangles with rounded corners, ovals, rectangles, or circles. Named with a short sentence, one word, or a phrase to express its essence.
- **Data Flow:** Describes the transfer of information between system parts. Represented by arrows. Named to identify the type of information being transferred.
- **Warehouse:** Storage for data to be used later. Represented by two horizontal lines. Can be data files, folders, optical discs, filing cabinets, etc. When data flows from the warehouse, it is considered reading; when data flows to the warehouse, it is called data entry or update.
- **Terminator:** An external entity that interacts with the system. Example:- Organizations like banks, customer groups, or different departments of the same organization that are not part of the system.

Symbols and Notations Used in DFDs

Two common systems of symbols are named after their creators:

- Yourdon and Coad
- Gane and Sarson

One main difference in their symbols is that Yourdon-Coad use circles for processes, while Gane and Sarson use rectangles with rounded corners, sometimes called lozenges. There are other symbol variations in use as well, so the important thing to keep in mind is to be clear and consistent in the shapes and notations you use to communicate and collaborate with others.

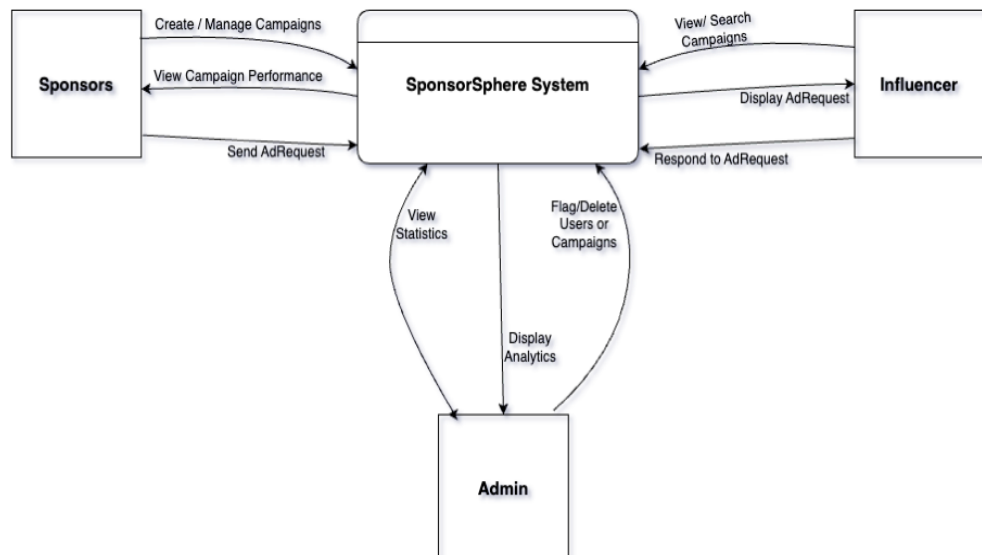
Using any convention's DFD rules or guidelines, the symbols depict the four components of data flow diagrams.

- **External Entity:** An outside system or person that interacts with the system, sending or receiving data. Symbol: A rectangle.
- **Process:** Transforms input data into output. Symbol: A rectangle with rounded corners or a circle.

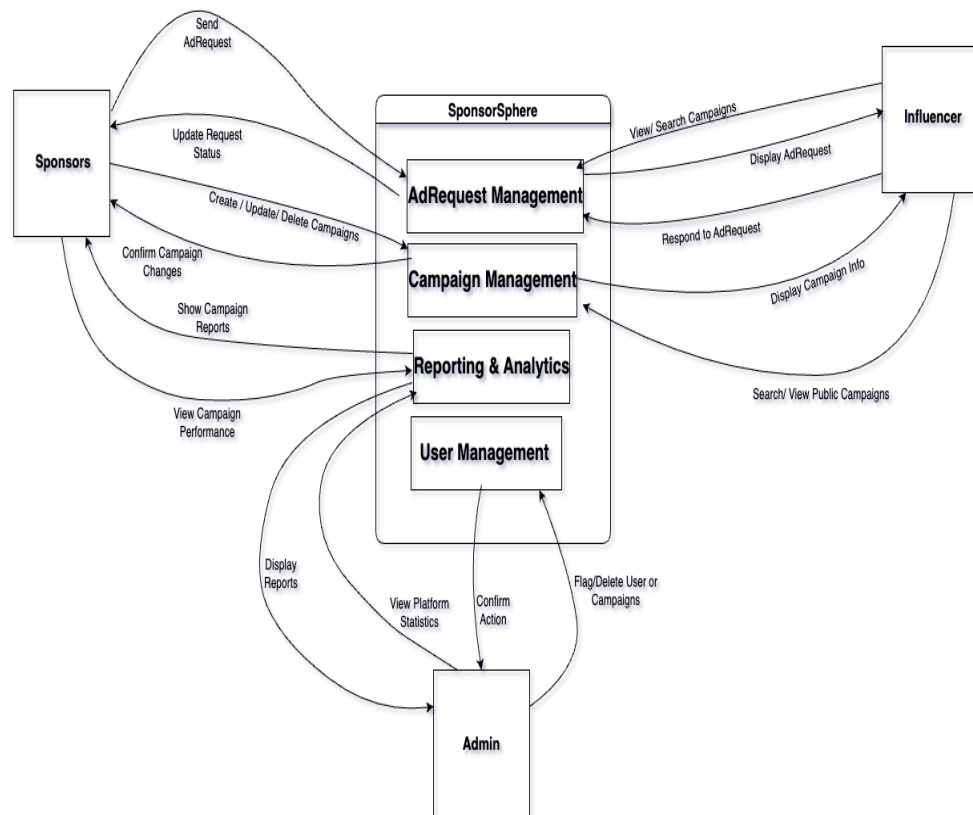
- **Data Store:** Repositories where data is stored for later use. Symbol: Two parallel horizontal lines.
- **Data Flow:** Routes data between external entities, processes, and data stores. Symbol: An arrow.



A. DFD (Level 0)



B. DFD (Level 1)



8.4 ER Diagram

ER diagram or Entity-Relationship Diagram (ERD) is a visual representation used in database design to illustrate the logical structure of a database. ERDs depict the entities, attributes, and relationships within a database, helping to model how data is organized and related to each other.

Components of ER Diagram

- **Entity:** An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.



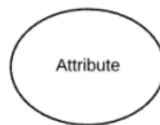
- Strong Entity:** A strong entity is an entity that can be uniquely identified by its own attributes. This means that each instance of the entity can be distinguished from all

other instances of the entity by its own attributes. The strong entity is represented as rectangle.

- ii) **Weak Entity:** An entity that depends on another entity called a weak entity. The weak entity doesn't contain any key attribute of its own. The weak entity is represented by a double rectangle.



- **Attributes:** The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.



- i. **Key Attribute:** The key attribute is used to represent the main characteristics of an entity. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.



- ii. **Composite Attribute:** An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.



- iii. **Multivalued attributes:** are those that are can take on more than one value. The double oval is used to represent multivalued attribute.



- iv. **Derived Attribute:** An attribute that can be derived from other attribute is known as a derived attribute. It can be represented by a dashed ellipse.



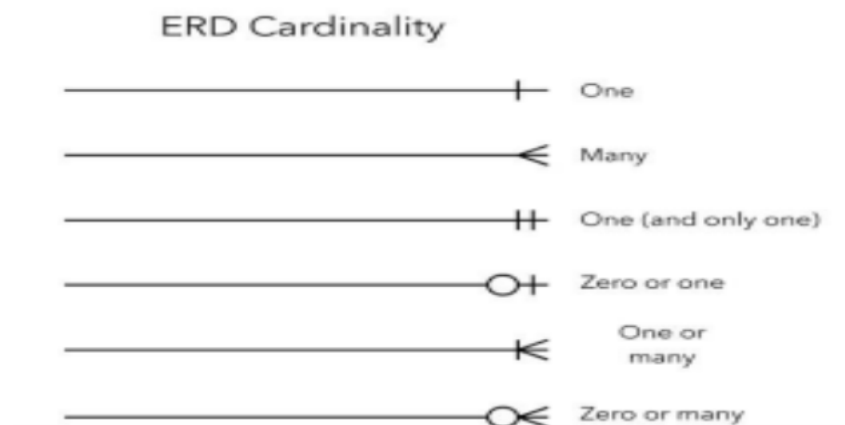
- **Relationship:** A relationship is used to describe the relation between entities. Diamond or rhombus is used to represent the relationship.



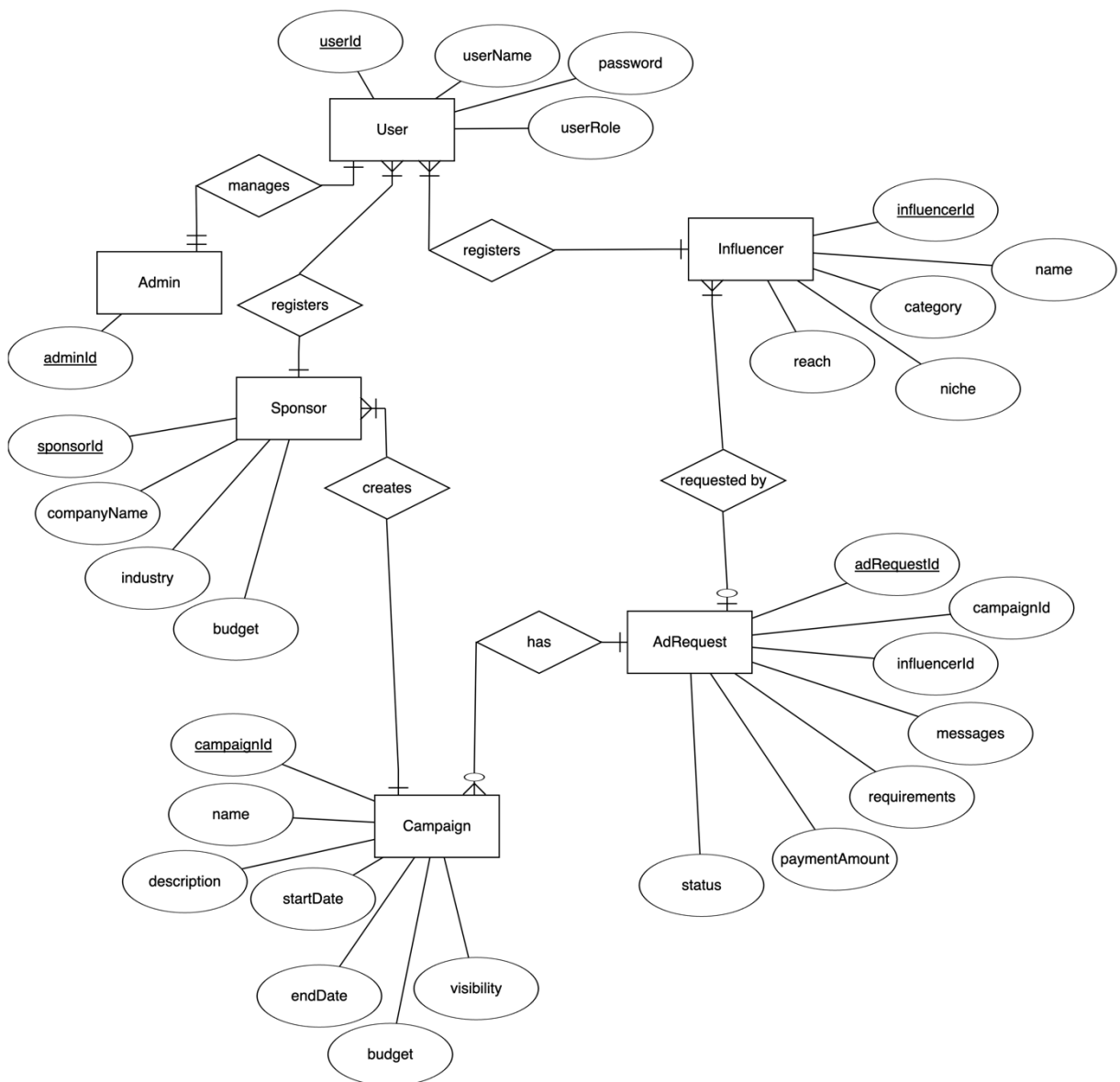
- Weak Relationships:** are connections between a weak entity and its owner.



- **Different types of cardinal relationships are:**
 - One-to-One Relationships
 - One-to-Many Relationships
 - May to One Relationships
 - Many-to-Many Relationships



A. ER Diagram (SponsorSphere)



8.5 Activity Diagram

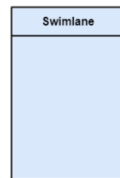
Activity diagram helps in envisioning the workflow from one activity to another. It put emphasis on the condition of flow and the order in which it occurs. The flow can be sequential, branched, or concurrent, and to deal with such kinds of flows, the activity diagram has come up with a fork, join, etc.

Components of Activity Diagram

- Activities:** The categorization of behavior into one or more actions is termed as an activity. In other words, it can be said that an activity is a network of nodes that are connected by edges. The edges depict the flow of execution. It may contain action nodes, control nodes, or object nodes.



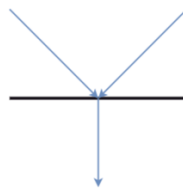
- **Activity partition /swimlane:** The swimlane is used to cluster all the related activities in one column or one row. It can be either vertical or horizontal. It used to add modularity to the activity diagram. It is not necessary to incorporate swimlane in the activity diagram. But it is used to add more transparency to the activity diagram.



- **Forks:** Forks and join nodes generate the concurrent flow inside the activity. A fork node consists of one inward edge and several outward edges. It is the same as that of various decision parameters. Whenever a data is received at an inward edge, it gets copied and split crossways various outward edges. It split a single inward flow into multiple parallel flows.



- **Join Nodes:** Join nodes are the opposite of fork nodes. A Logical AND operation is performed on all of the inward edges as it synchronizes the flow of input across one single output (outward) edge.



Notations of Activity Diagram

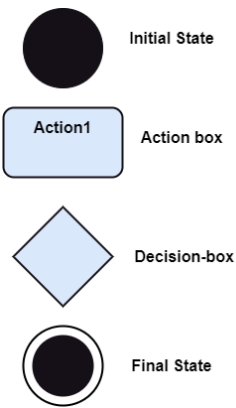
Activity diagram constitutes following notations:

Initial State: It depicts the initial stage or beginning of the set of actions.

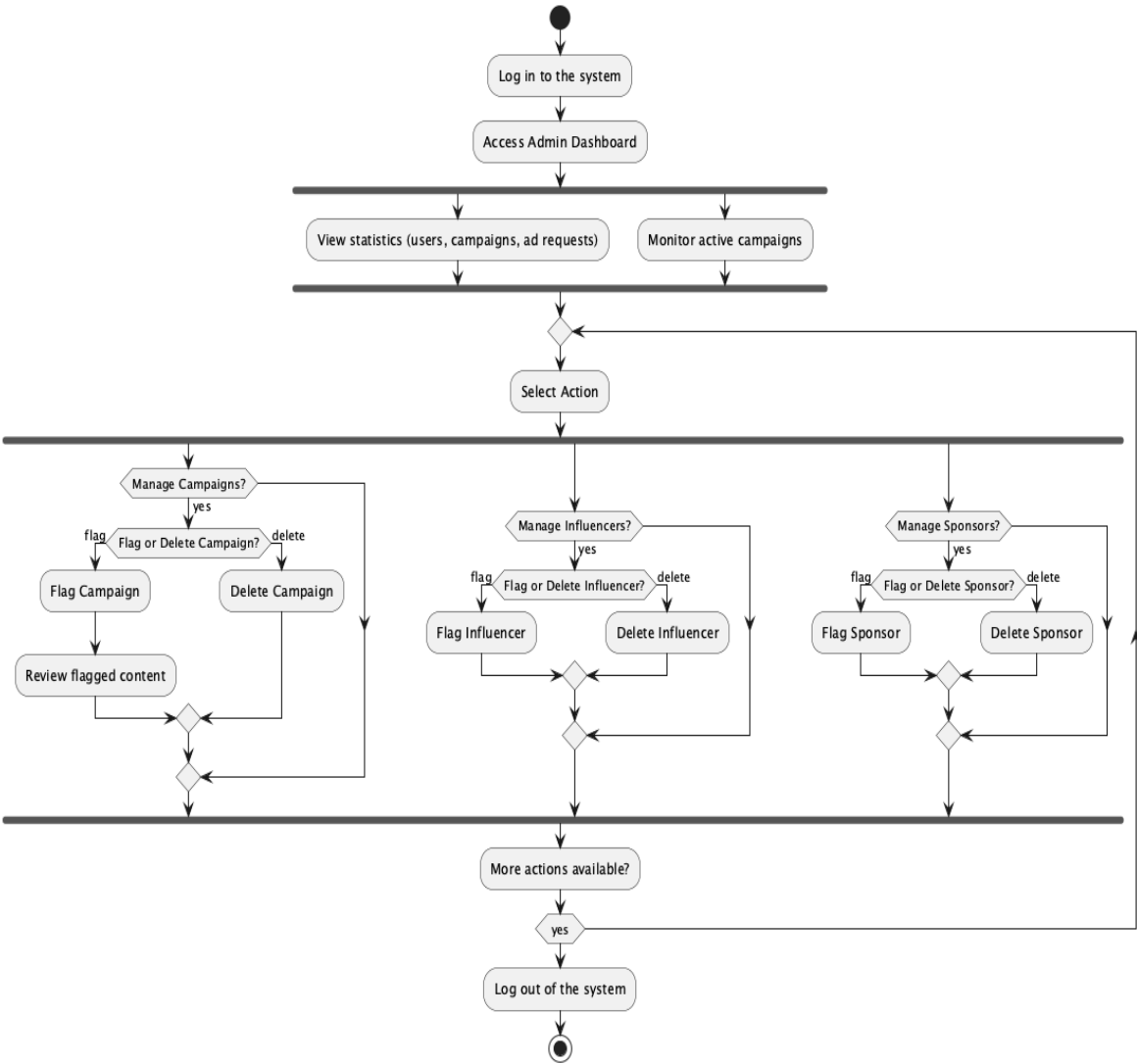
Final State: It is the stage where all the control flows and object flows end.

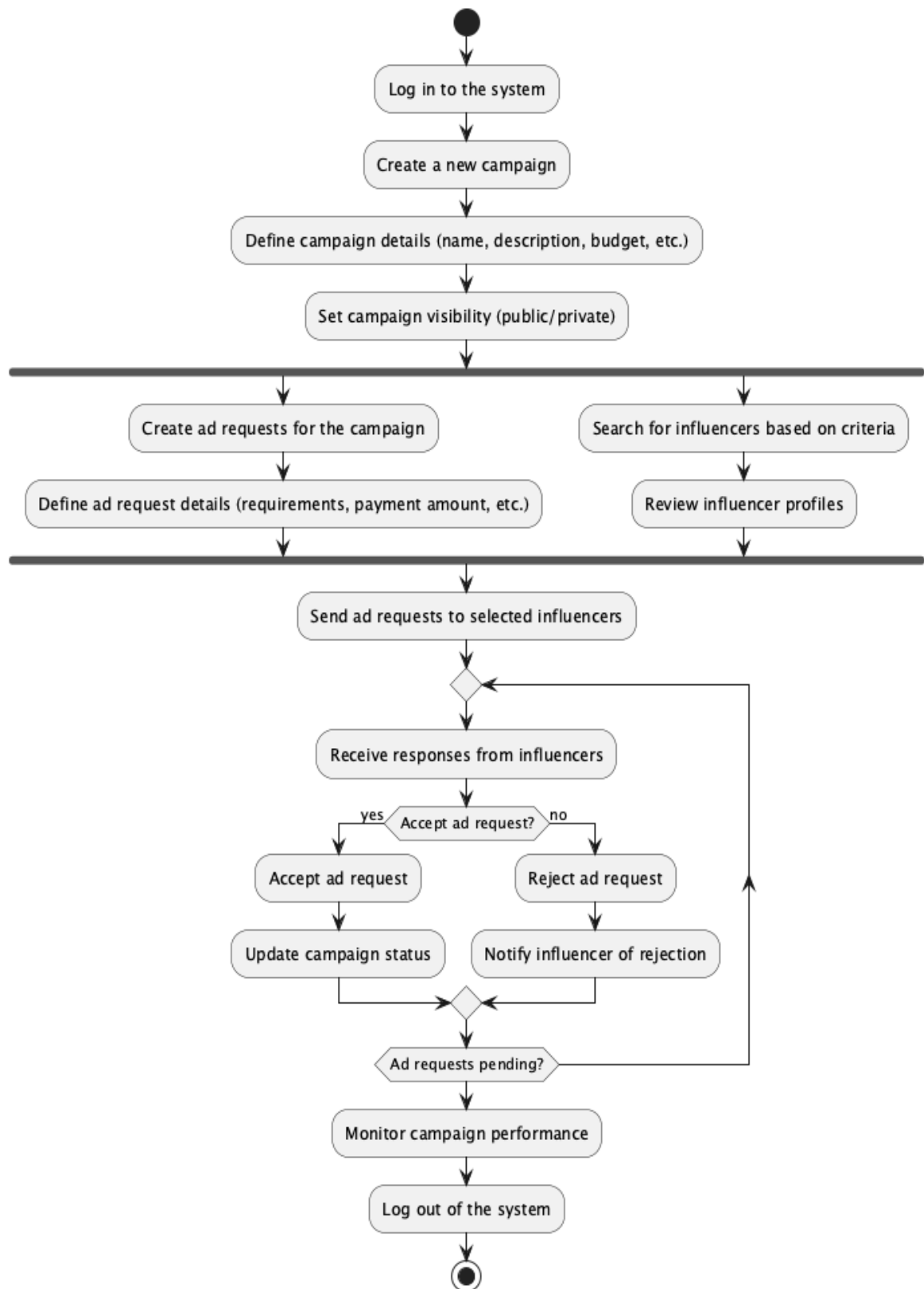
Decision Box: It makes sure that the control flow or object flow will follow only one path.

Action Box: It represents the set of actions that are to be performed.

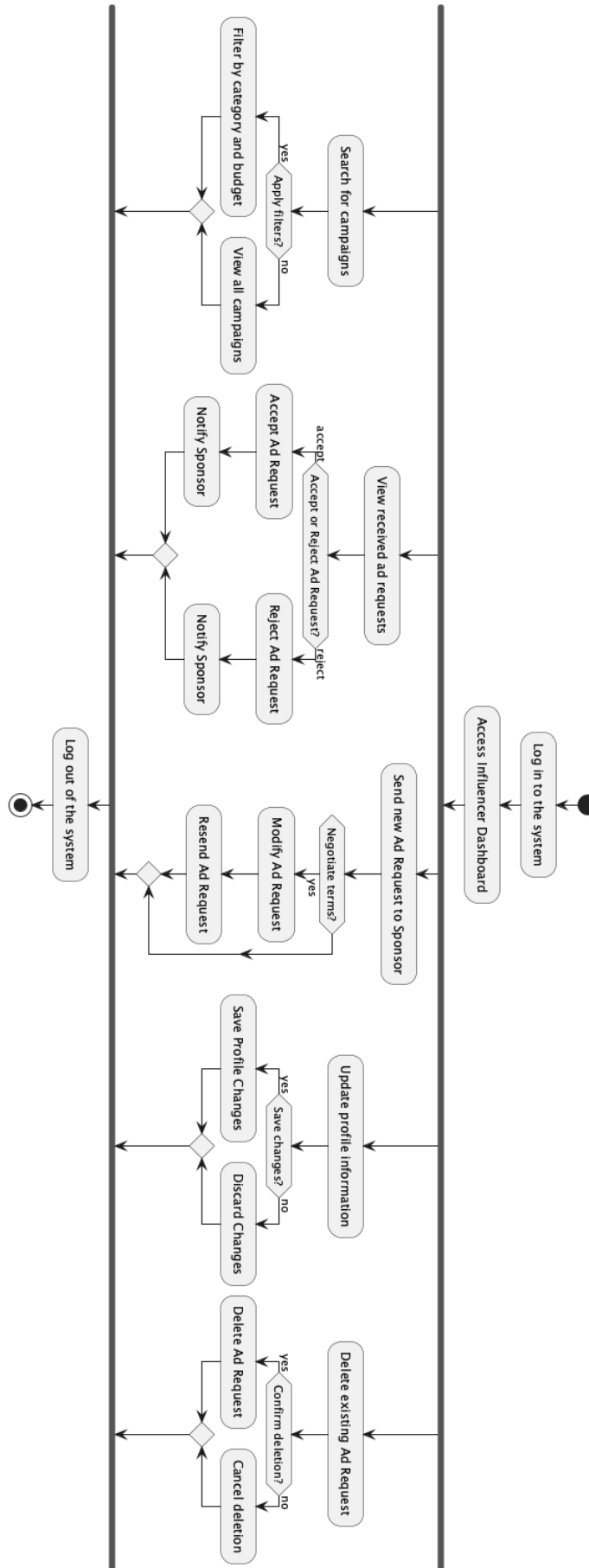


A. Activity Diagram (Admin)



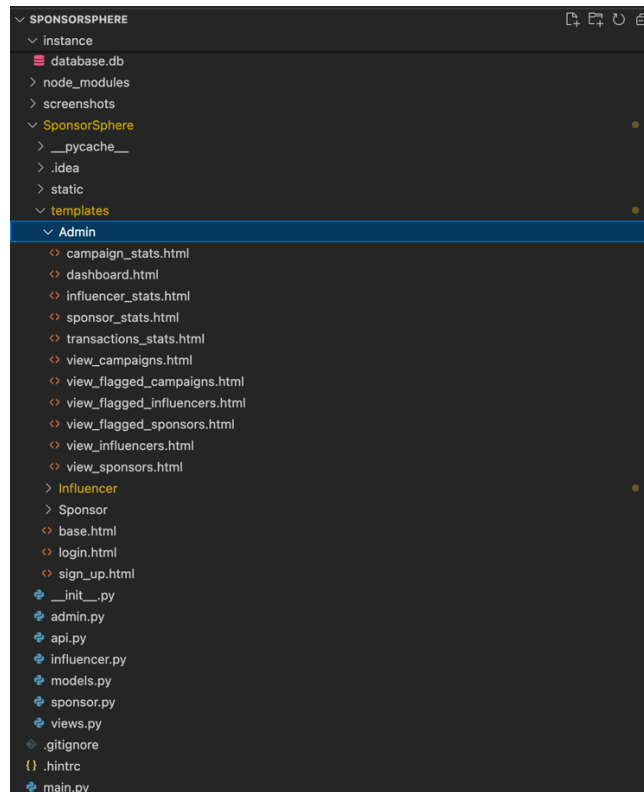
B. Activity Diagram (Sponsor)

C. Activity Diagram (Influencer)



9. Codes

9.1 Code File Structure



9.2 Codes

- **main.py**

```
from SponsorSphere import create_app
import os
app = create_app()
if __name__ == "__main__":
    port = int(
        os.environ.get("PORT", 5000)
    ) # Get the PORT environment variable, default to 5000 if not set
    app.run(
        host="0.0.0.0", port=port, debug=True
    ) # Run the app on all available network interfaces
```

- **__init__.py**

```
from flask import Flask
from flask_restful import Api, Resource
```

```
from flask_sqlalchemy import SQLAlchemy
from os import path
from flask_login import LoginManager
db = SQLAlchemy()
DB_NAME = "database.db"
def create_app():
    app = Flask(__name__)
    api = Api(app)
    app.config["SECRET_KEY"] = "idkra&w"
    app.config["UPLOAD_FOLDER"] = "Adfluence/static/uploads"
    app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:/// {DB_NAME}"
    db.init_app(app)
    from .views import views
    from .sponsor import sponsor
    from .admin import admin
    from .influencer import influencer
    app.register_blueprint(views, url_prefix="/")
    app.register_blueprint(admin, url_prefix="/admin")
    app.register_blueprint(sponsor, url_prefix="/sponsor")
    app.register_blueprint(influencer, url_prefix="/influencer")
    from .models import User
    with app.app_context():
        db.create_all()
    login_manager = LoginManager()
    login_manager.login_view = "views.login"
    login_manager.init_app(app)
    @login_manager.user_loader
    def load_user(id):
        return User.query.get(int(id))
    from .api import CampaignAPI
    from .api import AdRequestAPI
    from .api import UserAPI
    from .api import InfluencerAPI
    from .api import CampaignRequestAPI
    api.add_resource(UserAPI, "/api/user/<int:id>", "/api/user")
```

```

api.add_resource(CampaignAPI, "/api/campaign/<int:id>", "/api/campaign")
api.add_resource(AdRequestAPI, "/api/adrequest/<int:id>", "/api/adrequest")
api.add_resource(
    CampaignRequestAPI, "/api/CampaignRequest/<int:id>", "/api/CampaignRequest"
)
api.add_resource(InfluencerAPI, "/api/influencer/<int:id>", "/api/influencer")
return app
def create_database(app):
    if not path.exists("Adfluence/" + DB_NAME):
        db.create_all(app=app)
        print("Created Database!")

```

- **views.py**

```

from flask import Blueprint, render_template, request, flash, redirect, url_for
from .models import User
from werkzeug.security import generate_password_hash, check_password_hash
from . import db # means from __init__.py import db
from flask_login import login_user, login_required, logout_user, current_user
from functools import wraps
from flask import abort
views = Blueprint("views", __name__)
def role_required(role):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if current_user.role != role:
                abort(403)
            return f(*args, **kwargs)
        return decorated_function
    return decorator
# #####
# @views.route("/create-admin")
# def create_admin():
#     email = "admin@gmail.com"
#     password = "admin" # Set a secure password here

```

```

# role = "Admin"
# if not User.query.filter_by(email=email).first():
#     new_user = User(
#         email=email,
#         password=generate_password_hash(password, method="pbkdf2:sha256"),
#         name='Admin',
#         role='Admin',
#         niche='default_value', # Set a default niche here
#         flagged=0
#     )
#     db.session.add(new_user)
#     db.session.commit()
#     return "Admin user created successfully!"
# return "Admin user already exists!"
#####

@views.route("/", methods=["GET", "POST"])
def login():
    role = None
    if request.method == "POST":
        email = request.form.get("email")
        password = request.form.get("password")
        role = request.form.get("role")
        user = User.query.filter_by(email=email).first()
        if user:
            if check_password_hash(user.password, password):
                if (
                    user.role == role
                ): # Check if the user role matches the selected role
                    login_user(user, remember=True)
                    flash("Logged in successfully!", category="success")
                    if role == "Admin":
                        return redirect(url_for("admin.dashboard"))
                    elif role == "Sponsor":
                        return redirect(url_for("sponsor.dashboard"))
                    elif role == "Influencer":

```

```
        return redirect(url_for("influencer.dashboard"))
    else:
        flash(
            "Incorrect role selected. Please check your role.",
            category="error",
        )
    else:
        flash("Incorrect password, try again.", category="error")
    else:
        flash("Email does not exist.", category="error")
    return render_template("login.html", user=current_user, role = role)
@views.route("/logout")
@login_required
def logout():
    logout_user()
    return redirect(url_for("views.login"))
@views.route("/sign-up", methods=["GET", "POST"])
def sign_up():
    if request.method == "POST":
        email = request.form.get("email")
        name = request.form.get("firstName")
        password1 = request.form.get("password1")
        password2 = request.form.get("password2")
        role = request.form.get("role")
        niche = request.form.get("niche")
        user = User.query.filter_by(email=email).first()
        if user:
            flash("Email already exists.", category="error")
        elif len(email) < 4:
            flash("Email must be greater than 3 characters.", category="error")
        elif len(name) < 2:
            flash("First name must be greater than 1 character.", category="error")
        elif password1 != password2:
            flash("Passwords don't match.", category="error")
        elif len(password1) < 7:
```

```

        flash("Password must be at least 7 characters.", category="error")
    else:
        new_user = User(
            email=email,
            role=role,
            name=name,
            niche=niche,
            # password=generate_password_hash(password1),
            password = generate_password_hash(password1, method='pbkdf2:sha256')
        )
        db.session.add(new_user)
        db.session.commit()
        login_user(new_user, remember=True)
        flash("Account created!", category="success")
        return redirect(url_for("views.login"))
    return render_template("sign_up.html", user=current_user)

```

- **sponsor.py**

```

import csv
import os
from datetime import datetime
from uuid import uuid4
from sqlalchemy import desc
from flask import (
    Blueprint,
    render_template,
    request,
    flash,
    redirect,
    url_for,
    abort,
    send_file,
    current_app,
)
from flask_login import login_required, current_user

```

```

from sqlalchemy.sql import func
from . import db
from .models import (
    Campaign,
    AdRequest,
    Influencer,
    campaignRequest,
    Transaction,
    Rating,
    Negotiation,
)
from .views import role_required
sponsor = Blueprint("sponsor", __name__)
def get_allocated_budget(campaign_id):
    ad_requests_sum = (
        db.session.query(func.sum(AdRequest.payment_amount))
        .filter_by(campaign_id=campaign_id, status="Accepted")
        .scalar()
        or 0
    )
    campaign_requests_sum = (
        db.session.query(func.sum(campaignRequest.payment_amount))
        .filter_by(campaign_id=campaign_id, status="Accepted")
        .scalar()
        or 0
    )
    return ad_requests_sum + campaign_requests_sum
# ----- Sponsor Dashboard -----#
@sponsor.route("/dashboard", methods=["GET", "POST"])
@role_required("Sponsor")
@login_required
def dashboard():
    active_campaigns = Campaign.query.filter_by(user_id=current_user.id).all()
    campaign_requests = (
        campaignRequest.query.join(Campaign)

```

```
.filter(Campaign.user_id == current_user.id)
.order_by(desc(campaignRequest.id))
.all()
)
campaign_requests_with_details = []
for campaign_request in campaign_requests:
    campaign = Campaign.query.get(campaign_request.campaign_id)
    campaign_name = campaign.name
    influencer = Influencer.query.get(campaign_request.influencer_id)
    if influencer:
        influencer_name = influencer.name
        campaign_requests_with_details.append(
            {
                "campaign_name": campaign_name,
                "campaign_request": campaign_request,
                "influencer_name": influencer_name,
            }
        )
    else:
        flash("Influencer not found", category="error")
sent_requests = (
    AdRequest.query.join(Campaign)
    .filter(Campaign.user_id == current_user.id)
    .order_by(desc(AdRequest.id))
    .all()
)
sent_requests_with_details = []
for sent_request in sent_requests:
    influencer = Influencer.query.get(sent_request.influencer_id)
    influencer_name = influencer.name
    sent_requests_with_details.append(
        {"ad_request": sent_request, "influencer_name": influencer_name}
    )
return render_template(
    "Sponsor/dashboard.html",
```



```

        user=current_user,
        active_campaigns=active_campaigns,
        campaign_requests=campaign_requests_with_details,
        sent_requests=sent_requests_with_details,
    )
# ----- Campaign -----#
@sponsor.route("/createCampaign", methods=["GET", "POST"])
@role_required("Sponsor")
@login_required
def createCampaign():
    if request.method == "POST":
        name = request.form.get("name")
        description = request.form.get("description")
        start_date = request.form.get("start_date")
        end_date = request.form.get("end_date")
        budget = request.form.get("budget")
        visibility = request.form.get("visibility")
        goals = request.form.get("goals")
        niche = request.form.get("niche")
        user_id = current_user.id
        if visibility == "private" and niche == "All":
            flash("Private campaigns cannot be of All niche", "error")
            return redirect(url_for("sponsor.createCampaign"))
        new_campaign = Campaign(
            name=name,
            description=description,
            start_date=datetime.strptime(start_date, "%Y-%m-%d"),
            end_date=datetime.strptime(end_date, "%Y-%m-%d"),
            budget=budget,
            visibility=visibility,
            goals=goals,
            niche=niche,
            user_id=user_id,
        )
        db.session.add(new_campaign)

```

```
db.session.commit()
flash("Campaign created successfully!", category="success")
return redirect(url_for("sponsor.dashboard"))
return render_template("Sponsor/campaignForm.html", user=current_user)
@sponsor.route("/viewCampaign/<int:campaign_id>", methods=["GET"])
@role_required("Sponsor")
@login_required
def viewCampaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    campaignRequests = (
        campaignRequest.query.filter_by(campaign_id=campaign_id)
        .order_by(desc(campaignRequest.id))
        .all()
    )
    campaign_requests_with_details = []
    for campaign_request in campaignRequests:
        influencer = Influencer.query.get(campaign_request.influencer_id)
        if influencer:
            influencer_name = influencer.name
            campaign_requests_with_details.append(
                {
                    "request": campaign_request,
                    "influencer_name": influencer_name,
                    "request_type": "Received",
                }
            )
        else:
            flash("Influencer not found", category="error")
    adrequests = (
        AdRequest.query.filter_by(campaign_id=campaign_id)
        .order_by(desc(AdRequest.id))
        .all()
    )
    sent_requests_with_details = []
    for sent_request in adrequests:
```

```
adrequest_negotiations = sent_request.negotiations
influencer = Influencer.query.get(sent_request.influencer_id)
influencer_name = influencer.name
pending_found = any(nego.status == "Pending" for nego in adrequest_negotiations)
sent_requests_with_details.append(
    {
        "request": sent_request,
        "influencer_name": influencer_name,
        "request_type": "Sent",
        "negotiations": adrequest_negotiations,
        "pending_found": pending_found,
    }
)
return render_template(
    "Sponsor/viewCampaign.html",
    user=current_user,
    campaign=campaign,
    ad_requests=sent_requests_with_details,
    campaign_requests=campaign_requests_with_details,
)
@sponsor.route("/viewCampaigns", methods=["GET"])
@role_required("Sponsor")
@login_required
def viewCampaigns():
    active_campaigns = Campaign.query.filter(
        Campaign.user_id == current_user.id, Campaign.status == "Incomplete"
    ).all()
    return render_template(
        "Sponsor/viewCampaigns.html", user=current_user, campaigns=active_campaigns
    )
@sponsor.route("/deleteCampaign/<int:campaign_id>", methods=["POST"])
@role_required("Sponsor")
@login_required
def deleteCampaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
```

```
if campaign.user_id != current_user.id:
    flash("You are not authorized to delete this campaign.", category="error")
    return redirect(url_for("sponsor.viewCampaigns"))
db.session.delete(campaign)
db.session.commit()
flash("Campaign deleted successfully!", category="success")
return redirect(url_for("sponsor.viewCampaigns"))

@sponsor.route("/editCampaign/<int:campaign_id>", methods=["GET", "POST"])
@role_required("Sponsor")
@login_required
def editCampaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    # Ensure the current user is the owner of the campaign
    if campaign.user_id != current_user.id:
        flash("You do not have permission to edit this campaign.", category="error")
        return redirect(url_for("sponsor.viewCampaigns"))
    if request.method == "POST":
        name = request.form.get("name")
        description = request.form.get("description")
        start_date = request.form.get("start_date")
        end_date = request.form.get("end_date")
        budget = request.form.get("budget")
        visibility = request.form.get("visibility")
        goals = request.form.get("goals")
        # Check for missing fields
        if not all(
            [name, description, start_date, end_date, budget, visibility, goals]
        ):
            flash("All fields are required.", category="error")
            return render_template("Sponsor/editCampaign.html", campaign=campaign)
    try:
        campaign.name = name
        campaign.description = description
        campaign.start_date = datetime.strptime(start_date, "%Y-%m-%d")
        campaign.end_date = datetime.strptime(end_date, "%Y-%m-%d")
```

```
        campaign.budget = budget
        campaign.visibility = visibility
        campaign.goals = goals

        db.session.commit()
        flash("Campaign updated successfully!", category="success")
        return redirect(url_for("sponsor.viewCampaigns"))
    except ValueError as e:
        flash(f"Invalid input: {e}", category="error")
        return render_template("Sponsor/editCampaign.html", campaign=campaign)
    return render_template("Sponsor/editCampaign.html", campaign=campaign)

# mark campaign as completed
@sponsor.route("/campaign/<int:campaign_id>/mark_completed", methods=["POST"])
@role_required("Sponsor")
@login_required
def mark_completed(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    if campaign.user_id != current_user.id:
        flash(
            "You are not authorized to mark this campaign as completed.",
            category="error",
        )
        return redirect(url_for("sponsor.viewCampaigns"))
    campaign.status = "Completed"
    db.session.commit()
    flash("Campaign marked as completed successfully!", category="success")
    return redirect(url_for("sponsor.viewCampaign", campaign_id=campaign_id))

# view completed campaigns
@sponsor.route("/view_completed_campaigns")
@role_required("Sponsor")
@login_required
def view_completed_campaigns():
    campaigns = Campaign.query.filter_by(status="Completed").all()
    return render_template(
```

```

        "Sponsor/view_completed_campaigns.html",                campaigns=campaigns,
        user=current_user
    )
    # view completed campaign
    @sponsor.route("/view_completed_campaign/<int:campaign_id>")
    @role_required("Sponsor")
    @login_required
    def view_completed_campaign(campaign_id):
        campaign = Campaign.query.get_or_404(campaign_id)
        campaignRequests = (
            campaignRequest.query.filter_by(campaign_id=campaign_id)
            .order_by(desc(campaignRequest.id))
            .all()
        )
        campaign_requests_with_details = []
        for campaign_request in campaignRequests:
            influencer = Influencer.query.get(campaign_request.influencer_id)
            if influencer:
                influencer_name = influencer.name
                campaign_requests_with_details.append(
                    {
                        "request": campaign_request,
                        "influencer_name": influencer_name,
                        "request_type": "Received",
                    }
                )
            else:
                flash("Influencer not found", category="error")
        adrequests = (
            AdRequest.query.filter_by(campaign_id=campaign_id)
            .order_by(desc(AdRequest.id))
            .all()
        )
        sent_requests_with_details = []
        for sent_request in adrequests:

```

```

adrequest_negotiations = sent_request.negotiations
influencer = Influencer.query.get(sent_request.influencer_id)
influencer_name = influencer.name
pending_found = any(nego.status == "Pending" for nego in adrequest_negotiations)
sent_requests_with_details.append(
    {
        "request": sent_request,
        "influencer_name": influencer_name,
        "request_type": "Sent",
        "negotiations": adrequest_negotiations,
        "pending_found": pending_found,
    }
)
return render_template(
    "Sponsor/view_completed_campaign.html",
    campaign=campaign,
    user=current_user,
    ad_requests=sent_requests_with_details,
    campaign_requests=campaign_requests_with_details,
)
# -----Adrequest-----#
@sponsor.route("/campaign/<int:campaign_id>/create_ad_request", methods=["GET",
"POST"])
@role_required("Sponsor")
@login_required
def create_ad_request(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    if request.method == "POST":
        influencer_id = request.form["influencer_id"]
        messages = request.form["messages"]
        requirements = request.form["requirements"]
        payment_amount = request.form["payment_amount"]
        # Check if the new request would exceed the campaign budget
        allocated_budget = get_allocated_budget(campaign_id)
        allocated_budget = float(allocated_budget)

```

```

payment_amount = float(payment_amount)
if allocated_budget + payment_amount > campaign.budget:
    flash(
        f"This request would exceed the campaign budget. Available budget:
{campaign.budget - allocated_budget}",
        "error",
    )
    return redirect(
        url_for("sponsor.create_ad_request", campaign_id=campaign_id)
    )
influencer = Influencer.query.get(influencer_id)
if not influencer:
    flash("Influencer not found", "error")
    return redirect(url_for("sponsor.dashboard"))
ad_request = campaignRequest(
    campaign_id=campaign_id,
    influencer_id=influencer_id,
    messages=messages,
    requirements=requirements,
    payment_amount=payment_amount,
    status="Pending",
)
db.session.add(ad_request)
db.session.commit()
flash("Ad request created successfully.", category="success")
return redirect(url_for("sponsor.viewCampaign", campaign_id=campaign_id))
# getting all influencers
influencers = Influencer.query.all()
return render_template(
    "Sponsor/create_ad_request.html", campaign=campaign, influencers=influencers
)
@sponsor.route("/request_action/<int:request_id>/<action>", methods=["POST"])
@role_required("Sponsor")
@login_required
def request_action(request_id, action):

```



```

ad_request = campaignRequest.query.get_or_404(request_id)
if action == "accept":
    # Check if accepting this request would exceed the campaign budget
    allocated_budget = get_allocated_budget(ad_request.campaign_id)
    if allocated_budget + ad_request.payment_amount > ad_request.campaign.budget:
        flash(
            f"Accepting this request would exceed the campaign budget. Available budget:
{ad_request.campaign.budget - allocated_budget}",
            "error",
        )
        return redirect(url_for("sponsor.dashboard"))
    ad_request.status = "Accepted"
elif action == "reject":
    ad_request.status = "Rejected"
db.session.commit()
flash(f"Request has been {action}ed.", category="success")
return redirect(url_for("sponsor.dashboard"))
@sponsor.route("/ad_request/<int:ad_request_id>/delete", methods=["POST"])
@role_required("Sponsor")
@login_required
def delete_ad_request(ad_request_id):
    ad_request = AdRequest.query.get_or_404(ad_request_id)
    if not ad_request:
        ad_request = campaignRequest.query.get_or_404(ad_request_id)
    if ad_request.campaign.user_id != current_user.id:
        abort(403)
    db.session.delete(ad_request)
    db.session.commit()
    flash("Ad request deleted successfully.", category="success")
    return redirect(url_for("sponsor.dashboard"))
@sponsor.route("/view_completed_ad_requests/<int:campaign_id>")
@role_required("Sponsor")
@login_required
def view_completed_ad_requests(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)

```

```
campaignRequests = campaignRequest.query.filter_by(
    campaign_id=campaign_id, completion_confirmed=True
).all()
campaign_requests_with_details = []
for campaign_request in campaignRequests:
    influencer = Influencer.query.get(campaign_request.influencer_id)
    if influencer:
        influencer_name = influencer.name
        campaign_requests_with_details.append(
            {
                "request": campaign_request,
                "influencer_name": influencer_name,
                "request_type": "Received",
            }
        )
    else:
        flash("Influencer not found", category="error")
adrequests = AdRequest.query.filter_by(
    campaign_id=campaign_id, completion_confirmed=True
).all()
sent_requests_with_details = []
for sent_request in adrequests:
    influencer = Influencer.query.get(sent_request.influencer_id)
    influencer_name = influencer.name
    sent_requests_with_details.append(
        {
            "request": sent_request,
            "influencer_name": influencer_name,
            "request_type": "Sent",
        }
    )
return render_template(
    "Sponsor/view_completed_ad_requests.html",
    campaign=campaign,
    ad_requests=sent_requests_with_details,
```

```

        campaign_requests=campaign_requests_with_details,
    )
# -----Influencers-----#
@sponsor.route("/campaign/<int:campaign_id>/browse_influencers")
@login_required
@role_required("Sponsor")
def browse_influencers(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    # Get parameters from the request
    reach = request.args.get("reach")
    niche = request.args.get("niche")
    name = request.args.get("name")
    # Base query
    query = Influencer.query
    # Apply filters if provided
    if reach:
        query = query.filter(Influencer.reach >= int(reach))
    if niche:
        query = query.filter(Influencer.niche == niche)
    if name:
        query = query.filter(Influencer.name.ilike(f"%{name}%"))
    # Execute the query
    filtered_influencers = query.all()
    return render_template(
        "Sponsor/browse_influencers.html",
        influencers=filtered_influencers,
        campaign=campaign,
    )
@sponsor.route("/view_all_influencers/<int:campaign_id>")
@role_required("Sponsor")
def view_all_influencers(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    influencers = Influencer.query.all()
    return render_template(
        "Sponsor/browse_influencers.html", influencers=influencers, campaign=campaign

```

```

    )
@sponsor.route(
    "/viewCampaign/<int:campaign_id>/<int:ad_request_id>/confirm_completion",
    methods=["POST"],
)
@role_required("Sponsor")
@login_required
def confirm_completion(campaign_id, ad_request_id):
    request_type = request.form.get("request_type")
    if request_type == "Sent":
        ad_request = AdRequest.query.get_or_404(ad_request_id)
        if ad_request.completed == True:
            ad_request.completion_confirmed = True
        if request_type == "Received":
            campaign_request = campaignRequest.query.get_or_404(ad_request_id)
            if campaign_request.completed == True:
                campaign_request.completion_confirmed = True
    db.session.commit()
    flash("Ad request completed successfully.", category="success")
    return redirect(url_for("sponsor.viewCampaign", campaign_id=campaign_id))

# -----Payment Gateway-----#
@sponsor.route("/payment_history/<int:user_id>")
@role_required("Sponsor")
@login_required
def payment_history(user_id):
    transactions = Transaction.query.filter_by(user_id=user_id).all()
    transaction_with_details = []
    for transaction in transactions:
        influencer = Influencer.query.get(transaction.influencer_id)
        influencer_name = influencer.name
        influencer_platform = influencer.platform
        request_type = transaction.request_type
        if request_type == "Sent":
            ad_request = AdRequest.query.get(transaction.ad_request_id)

```

```

elif request_type == "Received":
    ad_request = campaignRequest.query.get(transaction.ad_request_id)

    campaign_id = ad_request.campaign_id
    campaign = Campaign.query.get(campaign_id)
    campaign_name = campaign.name
    transaction_with_details.append(
        {
            "transaction": transaction,
            "influencer_name": influencer_name,
            "influencer_platform": influencer_platform,
            "campaign_name": campaign_name,
        }
    )
return render_template(
    "sponsor/payment_history.html", transactions=transaction_with_details
)

@sponsor.route("/make_payment/<int:ad_request_id>", methods=["GET", "POST"])
@role_required("Sponsor")
@login_required
def make_payment(ad_request_id):
    request_type = request.args.get("request_type")
    print(request_type)
    if request_type == "Sent":
        ad_request = AdRequest.query.get_or_404(ad_request_id)
    if request_type == "Received":
        ad_request = campaignRequest.query.get_or_404(ad_request_id)

    influencer_id = ad_request.influencer_id
    influencer = Influencer.query.get_or_404(influencer_id)
    print(influencer.name)
    adrequest_with_details = {"request": ad_request, "influencer_name":
influencer.name}
    if request.method == "GET":

```

```
        return render_template(
            "sponsor/make_payment.html", ad_request=adrequest_with_details
        )
    if request.method == "POST":
        card_number = request.form.get("card_number")
        expiration_date = request.form.get("expiration_date")
        cvv = request.form.get("cvv")
        # Validate card details
        if not card_number or len(card_number) != 16 or not card_number.isdigit():
            flash(
                "Invalid card number.Card number must be 16 digits.", category="error"
            )
        influencer_id = ad_request.influencer_id
        # Dummy payment processing
        transaction = Transaction(
            ad_request_id=ad_request.id,
            influencer_id=influencer_id,
            user_id=current_user.id,
            request_type=request_type,
            amount=ad_request.payment_amount,
            date=func.now(),
            status=True,
        )
        db.session.add(transaction)
        db.session.commit()
        print(ad_request.payment_done)
        ad_request.payment_done = True
        influencer = Influencer.query.get_or_404(influencer_id)
        influencer.bank_account_balance += ad_request.payment_amount
        db.session.commit()
        flash("Payment successful!", category="success")
        return redirect(url_for("sponsor.viewCampaigns"))
    return render_template(
        "sponsor/make_payment.html", ad_request=adrequest_with_details
    )
```

```

# -----Export CSV-----#
@sponsor.route("/export_csv")
@role_required("Sponsor")
@login_required
def export_csv():
    user_id = current_user.id
    transactions = Transaction.query.filter_by(user_id=user_id).all()
    transaction_with_details = []
    for transaction in transactions:
        influencer = Influencer.query.get(transaction.influencer_id)
        influencer_name = influencer.name
        influencer_platform = influencer.platform
        request_type = transaction.request_type
        if request_type == "Sent":
            ad_request = AdRequest.query.get(transaction.ad_request_id)
        elif request_type == "Received":
            ad_request = campaignRequest.query.get(transaction.ad_request_id)

        campaign_id = ad_request.campaign_id
        campaign = Campaign.query.get(campaign_id)
        campaign_name = campaign.name
        transaction_with_details.append(
            {
                "transaction": transaction,
                "influencer_name": influencer_name,
                "influencer_platform": influencer_platform,
                "campaign_name": campaign_name,
            }
        )
    filename = uuid4().hex + ".csv"
    url = "static/csv/" + filename
    filepath = os.path.join(current_app.root_path, "static", "csv", filename)
    Sno = 0

    # Ensure the directory exists

```

```

os.makedirs(os.path.dirname(filepath), exist_ok=True)

with open(filepath, "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(
        ["Sno.", "InfluencerName", "Platform", "CampaignName", "RequestType", "Amount", "Date", ]
    )
    for transaction in transaction_with_details:
        Sno += 1
        writer.writerow(
            [
                Sno,
                transaction["influencer_name"],
                transaction["influencer_platform"],
                transaction["campaign_name"],
                transaction["transaction"].request_type,
                transaction["transaction"].amount,
                transaction["transaction"].date,
            ]
        )

    return send_file(filepath, as_attachment=True, download_name=filename)

# -----Ratings-----#
@sponsor.route("/view_ratings")
@role_required("Sponsor")
@login_required
def view_ratings():
    rater_id = current_user.id
    ratings = Rating.query.filter_by(rater_id=rater_id).all()
    ratings_with_details = []
    for rating in ratings:
        influencer = Influencer.query.get(rating.ratee_id)
        influencer_name = influencer.name
        ratings_with_details.append(
            {

```



```
        "rating": rating,
        "influencer_name": influencer_name,
    }
)
return render_template("sponsor/view_ratings.html", ratings=ratings_with_details)
@sponsor.route(
    "rate_influencer/<int:ad_request_id>/<string:request_type>", methods=["GET",
"POST"]
)
@login_required
def rate_influencer(ad_request_id, request_type):
    transaction = Transaction.query.get_or_404(
        ad_request_id,
    )
    if request_type == "Sent":
        ad_request = AdRequest.query.get_or_404(ad_request_id)
    else:
        ad_request = None
    if request_type == "Received":
        ad_request = campaignRequest.query.get_or_404(ad_request_id)
    if transaction.user_id != current_user.id:
        flash("You are not authorized to rate this transaction.", "danger")
        return redirect(url_for("sponsor.dashboard"))
    influencer = Influencer.query.get_or_404(transaction.influencer_id)
    if request.method == "POST":
        rating_value = request.form.get("rating")
        review = request.form.get("review")
        new_rating = Rating(
            transaction_id=transaction.id,
            rater_id=current_user.id,
            ratee_id=transaction.influencer_id,
            rating=rating_value,
            review=review,
        )
```

```

        db.session.add(new_rating)
        ad_request.rating_done = True
        db.session.commit()
        flash("Rating submitted successfully.", "success")
        return redirect(url_for("sponsor.dashboard"))
    return render_template(
        "sponsor/rate_influencer.html",
        transaction=transaction,
        influencer_name=influencer.name,
    )
@sponsor.route(
    "/viewRequest/<string:request_type>/<int:ad_request_id>",      methods=["GET",
"POST"]
)
@role_required("Sponsor")
@login_required
def viewRequest(request_type, ad_request_id):
    if request_type == "Sent":
        ad_request = AdRequest.query.get_or_404(ad_request_id)
        # get negotiation details from ad_request
        nego_details = ad_request.negotiations
    else:
        ad_request = None
    if request_type == "Received":
        ad_request = campaignRequest.query.get_or_404(ad_request_id)
        nego_details = ad_request.negotiations
    print(nego_details)
    return render_template(
        "sponsor/viewRequest.html", ad_request=ad_request, nego_details=nego_details
    )
# ----- Accept Negotiation ----- #
@sponsor.route("/accept_negotiation/<int:negotiation_id>", methods=["POST"])
@role_required("Sponsor")
@login_required
def accept_negotiation(negotiation_id):

```

```

negotiation = Negotiation.query.get_or_404(negotiation_id)
ad_request_id = negotiation.ad_request_id
negotiation.status = "Accepted"
db.session.commit()

flash("Negotiation accepted successfully.", category="success")
return redirect(url_for("sponsor.viewCampaigns"))

# ----- Reject Negotiation ----- #
@sponsor.route("/reject_negotiation/<int:negotiation_id>", methods=["POST"])
@role_required("Sponsor")
@login_required
def reject_negotiation(negotiation_id):
    negotiation = Negotiation.query.get_or_404(negotiation_id)
    ad_request_id = negotiation.ad_request_id
    negotiation.status = "Rejected"
    db.session.commit()
    flash("Negotiation rejected successfully.", category="success")
    return redirect(url_for("sponsor.viewCampaigns"))

```

- **admin.py**

```

from flask import Blueprint, render_template, request, flash, redirect, url_for
from flask_login import login_required
from sqlalchemy.sql import func, extract
from . import db # means from __init__.py import db
from .models import (
    User, Influencer, AdRequest, Campaign, campaignRequest, Bookmark, Transaction, Rating,
)
from .views import role_required
admin = Blueprint("admin", __name__)
@admin.route("/dashboard")
@role_required("Admin")
@login_required
def dashboard():
    user_count = User.query.count()
    influencer_count = Influencer.query.count()
    sponsor_count = user_count - influencer_count

```

```

campaign_count = Campaign.query.count()
transaction_count = Transaction.query.count()
total_transaction_amount = (
    db.session.query(db.func.sum(Transaction.amount)).scalar() or 0
)
average_transaction_amount = (
    db.session.query(db.func.avg(Transaction.amount)).scalar() or 0
)
ad_request_count = AdRequest.query.count()
average_influencer_balance = (
    db.session.query(db.func.avg(Influencer.bank_account_balance)).scalar() or 0
)
average_rating = db.session.query(db.func.avg(Rating.rating)).scalar() or 0
campaigns_by_status = (
    db.session.query(Campaign.status, db.func.count(Campaign.id))
    .group_by(Campaign.status)
    .all()
)
labels_campaign_by_status = [campaign[0] for campaign in campaigns_by_status]
values_campaign_by_status = [campaign[1] for campaign in campaigns_by_status]
return render_template(
    "Admin/dashboard.html",
    user_count=user_count,
    influencer_count=influencer_count,
    sponsor_count=sponsor_count,
    campaign_count=campaign_count,
    transaction_count=transaction_count,
    total_transaction_amount=total_transaction_amount,
    average_transaction_amount=average_transaction_amount,
    ad_request_count=ad_request_count,
    average_influencer_balance=average_influencer_balance,
    average_rating=average_rating,
    labels_campaign_by_status=labels_campaign_by_status,
    values_campaign_by_status=values_campaign_by_status,
)

```

```

@admin.route("/influencer_stats")
@role_required("Admin")
@login_required
def influencer_stats():
    influencer_count = Influencer.query.count()
    most_used_platform = (
        db.session.query(Influencer.platform, func.count(Influencer.platform))
        .group_by(Influencer.platform)
        .order_by(func.count(Influencer.platform).desc())
        .first()
    )
    most_populated_niche = (
        db.session.query(Influencer.niche, func.count(Influencer.niche))
        .group_by(Influencer.niche)
        .order_by(func.count(Influencer.niche).desc())
        .first()
    )
    average_influencer_balance = (
        db.session.query(db.func.avg(Influencer.bank_account_balance)).scalar() or 0
    )
    influencers_by_niche = (
        db.session.query(Influencer.niche, func.count(Influencer.niche))
        .group_by(Influencer.niche)
        .all()
    )
    influencers_by_niche_labels = [influencer[0] for influencer in influencers_by_niche]
    influencers_by_niche_values = [influencer[1] for influencer in influencers_by_niche]
    influencers_by_platform = (
        db.session.query(Influencer.platform, func.count(Influencer.platform))
        .group_by(Influencer.platform)
        .all()
    )
    influencers_by_platform_labels = [
        influencer[0] for influencer in influencers_by_platform
    ]

```

```

influencers_by_platform_values = [
    influencer[1] for influencer in influencers_by_platform
]
average_influencer_reach = (
    db.session.query(db.func.avg(Influencer.reach)).scalar() or 0
)
return render_template(
    "Admin/influencer_stats.html",
    influencer_count=influencer_count,
    most_used_platform=most_used_platform,
    most_populated_niche=most_populated_niche,
    average_influencer_balance=average_influencer_balance,
    influencers_by_niche_labels=influencers_by_niche_labels,
    influencers_by_niche_values=influencers_by_niche_values,
    influencers_by_platform=influencers_by_platform,
    average_influencer_reach=average_influencer_reach,
    influencers_by_platform_labels=influencers_by_platform_labels,
    influencers_by_platform_values=influencers_by_platform_values,
)
@admin.route("/sponsor_stats")
@role_required("Admin")
@login_required
def sponsor_stats():
    influencer_count = Influencer.query.count()
    user_count = User.query.count()
    sponsor_count = user_count - influencer_count
    most_populated_niche = (
        db.session.query(User.niche, func.count(User.niche))
        .filter(User.role == "Sponsor")
        .group_by(User.niche)
        .order_by(func.count(User.niche).desc())
        .first()
    )
    sponsor_by_niche = (
        db.session.query(User.niche, func.count(User.niche))

```

```

        .filter(User.role == "Sponsor")
        .group_by(User.niche)
        .all()
    )
    sponsor_by_niche_labels = [sponsor[0] for sponsor in sponsor_by_niche]
    sponsor_by_niche_values = [sponsor[1] for sponsor in sponsor_by_niche]
    transaction_count = Transaction.query.count()
    total_transaction_amount = (
        db.session.query(db.func.sum(Transaction.amount)).scalar() or 0
    )
    average_transaction_amount = (
        db.session.query(db.func.avg(Transaction.amount)).scalar() or 0
    )
    transaction_by_request_type = (
        db.session.query(Transaction.request_type, func.count(Transaction.request_type))
        .group_by(Transaction.request_type)
        .all()
    )
    transaction_by_request_type_labels = [
        transaction[0] for transaction in transaction_by_request_type
    ]
    transaction_by_request_type_values = [
        transaction[1] for transaction in transaction_by_request_type
    ]
    return render_template(
        "Admin/sponsor_stats.html",
        most_populated_niche=most_populated_niche,
        sponsor_count=sponsor_count,
        transaction_count=transaction_count,
        total_transaction_amount=total_transaction_amount,
        average_transaction_amount=average_transaction_amount,
        transaction_by_request_type=transaction_by_request_type,
        transaction_by_request_type_labels=transaction_by_request_type_labels,
        transaction_by_request_type_values=transaction_by_request_type_values,
        sponsor_by_niche=sponsor_by_niche,

```

```

        sponsor_by_niche_labels=sponsor_by_niche_labels,
        sponsor_by_niche_values=sponsor_by_niche_values,
    )
@admin.route("/campaign_stats")
@role_required("Admin")
@login_required
def campaign_stats():
    campaign_count = Campaign.query.count()
    campaigns_by_visibility = (
        db.session.query(Campaign.visibility, db.func.count(Campaign.id))
        .group_by(Campaign.visibility)
        .all()
    )
    campaign_by_visibility_labels = [
        campaign[0] for campaign in campaigns_by_visibility
    ]
    campaign_by_visibility_values = [
        campaign[1] for campaign in campaigns_by_visibility
    ]
    most_populated_niches = (
        db.session.query(Campaign.niche, func.count(Campaign.niche))
        .group_by(Campaign.niche)
        .order_by(func.count(Campaign.niche).desc())
        .limit(2)
        .all()
    )
    # Extracting the first and second values
    if len(most_populated_niches) > 1:
        first_most_populated_niche = most_populated_niches[0]
        second_most_populated_niche = most_populated_niches[1]
    elif len(most_populated_niches) == 1:
        first_most_populated_niche = most_populated_niches[0]
        second_most_populated_niche = None
    else:
        first_most_populated_niche = None

```



```
second_most_populated_niche = None
campaigns_by_niche = (
    db.session.query(Campaign.niche, db.func.count(Campaign.id))
    .group_by(Campaign.niche)
    .all()
)
campaign_by_niche_labels = [campaign[0] for campaign in campaigns_by_niche]
campaign_by_niche_values = [campaign[1] for campaign in campaigns_by_niche]
# Fetch ad_requests data
ad_requests_data = (
    db.session.query(AdRequest.status, func.count(AdRequest.id))
    .group_by(AdRequest.status)
    .all()
)
# Fetch campaign_requests data
campaign_requests_data = (
    db.session.query(campaignRequest.status, func.count(campaignRequest.id))
    .group_by(campaignRequest.status)
    .all()
)
# Combine both data sets
combined_requests_data = {}
for status, count in ad_requests_data + campaign_requests_data:
    if status in combined_requests_data:
        combined_requests_data[status] += count
    else:
        combined_requests_data[status] = count
ads_by_status_labels = list(combined_requests_data.keys())
ads_by_status_values = list(combined_requests_data.values())
ad_requests_data_1 = (
    db.session.query(AdRequest.payment_done, func.count(AdRequest.id))
    .group_by(AdRequest.payment_done)
    .all()
)
campaign_requests_data_1 = (
```

```

        db.session.query(campaignRequest.payment_done, func.count(campaignRequest.id))
        .group_by(campaignRequest.payment_done)
        .all()
    )
    combined_requests_data_1 = {}
    for status, count in ad_requests_data_1 + campaign_requests_data_1:
        if status in combined_requests_data_1:
            combined_requests_data_1[status] += count
        else:
            combined_requests_data_1[status] = count
    status_mapping = {False: "Payment Pending", True: "Payment Done"}
    renamed_combined_requests_data_1 = {
        status_mapping[status]: count
        for status, count in combined_requests_data_1.items()
    }
    ads_by_status_labels_1 = list(renamed_combined_requests_data_1.keys())
    ads_by_status_values_1 = list(renamed_combined_requests_data_1.values())

    average_campaign_budget = (
        db.session.query(func.round(func.avg(Campaign.budget), 2).scalar() or 0
    )
    ad_request_count = db.session.query(func.count(AdRequest.id)).scalar() or 0
    campaign_request_count = (
        db.session.query(func.count(campaignRequest.id)).scalar() or 0
    )
    total_request_count = ad_request_count + campaign_request_count
    total_budget = db.session.query(func.sum(Campaign.budget)).scalar() or 0
    bookmark_count = Bookmark.query.count()
    most_popular_influencer_niche = (
        db.session.query(Influencer.niche, func.count(Influencer.niche))
        .group_by(Influencer.niche)
        .order_by(func.count(Influencer.niche).desc())
        .first()
    )
    return render_template(

```

```

"Admin/campaign_stats.html",
total_request_count=total_request_count,
total_budget=total_budget,
first_most_populated_niche=first_most_populated_niche,
second_most_populated_niche=second_most_populated_niche,
most_popular_influencer_niche=most_popular_influencer_niche,
campaign_count=campaign_count,
bookmark_count=bookmark_count,
campaigns_by_visibility=campaigns_by_visibility,
campaigns_by_niche=campaigns_by_niche,
average_campaign_budget=average_campaign_budget,
ads_by_status_labels=ads_by_status_labels,
ads_by_status_values=ads_by_status_values,
campaign_by_visibility_labels=campaign_by_visibility_labels,
campaign_by_visibility_values=campaign_by_visibility_values,
campaign_by_niche_labels=campaign_by_niche_labels,
campaign_by_niche_values=campaign_by_niche_values,
ads_by_payment_labels=ads_by_status_labels_1,
ads_by_payment_values=ads_by_status_values_1,
)
@admin.route("view_campaigns", methods=["GET", "POST"])
@role_required("Admin")
@login_required
def view_campaigns():
    if request.method == "POST":
        search_query = request.form.get("search_query")
        campaigns = Campaign.query.filter(
            Campaign.name.ilike(f"%{search_query}%")
        ).all()
    else:
        campaigns = Campaign.query.all()
    return render_template("Admin/view_campaigns.html", campaigns=campaigns)
@admin.route("/flag_campaign/<int:campaign_id>", methods=["POST"])
@role_required("Admin")
@login_required

```

```
def flag_campaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    campaign.flagged = True
    db.session.commit()
    flash("Campaign flagged!", category="success")
    return redirect(url_for("admin.view_campaigns"))
@admin.route("/unflag_campaign/<int:campaign_id>", methods=["POST"])
@role_required("Admin")
@login_required
def unflag_campaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    campaign.flagged = False
    db.session.commit()
    flash("Campaign unflagged!", category="success")
    return redirect(url_for("admin.view_flagged_campaigns"))
@admin.route("/delete_campaign/<int:campaign_id>", methods=["POST"])
@role_required("Admin")
@login_required
def delete_campaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    db.session.delete(campaign)
    db.session.commit()
    flash("Campaign deleted!", category="success")
    return redirect(url_for("admin.view_flagged_campaigns"))
@admin.route("/view_flagged_campaigns")
@role_required("Admin")
@login_required
def view_flagged_campaigns():
    campaigns = Campaign.query.filter_by(flagged=True).all()
    return render_template("Admin/view_flagged_campaigns.html",
campaigns=campaigns)
@admin.route("/view_influencers", methods=["GET", "POST"])
@role_required("Admin")
@login_required
def view_influencers():
```

```
# Get parameters from the request
reach = request.args.get("reach")
niche = request.args.get("niche")
# Base query
query = Influencer.query
# Apply filters if provided
if reach:
    query = query.filter(Influencer.reach >= int(reach))
if niche:
    query = query.filter(Influencer.niche == niche)
# Execute the query
filtered_influencers = query.all()
return render_template(
    "Admin/view_influencers.html", influencers=filtered_influencers
)
@admin.route("/view_all_influencers")
@role_required("Admin")
@login_required
def view_all_influencers():
    influencers = Influencer.query.all()
    return render_template("Admin/view_influencers.html", influencers=influencers)
@admin.route("/flag_influencer/<int:influencer_id>", methods=["POST"])
@role_required("Admin")
@login_required
def flag_influencer(influencer_id):
    influencer = Influencer.query.get_or_404(influencer_id)
    influencer.flagged = True
    db.session.commit()
    flash("Influencer flagged!", category="success")
    return redirect(url_for("admin.view_influencers"))
@admin.route("/unflag_influencer/<int:influencer_id>", methods=["POST"])
@role_required("Admin")
@login_required
def unflag_influencer(influencer_id):
    influencer = Influencer.query.get_or_404(influencer_id)
```

```
influencer.flagged = False
db.session.commit()
flash("Influencer Reinstated!", category="success")
return redirect(url_for("admin.view_flagged_influencers"))
@admin.route("/delete_influencer/<int:influencer_id>", methods=["POST"])
@role_required("Admin")
@login_required
def delete_influencer(influencer_id):
    influencer = Influencer.query.get_or_404(influencer_id)
    with db.session.no_autoflush:
        for rating in influencer.ratings:
            db.session.delete(rating)
        for transaction in influencer.transactions:
            transaction.influencer_id = None
            db.session.delete(transaction)
        for adrequest in influencer.adrequests:
            if adrequest:
                db.session.delete(adrequest)
        for campaignrequest in influencer.campaignrequests:
            if campaignrequest:
                db.session.delete(campaignrequest)
    user = User.query.get(influencer.user_id)
    db.session.delete(user)
    db.session.delete(influencer)
    db.session.commit()
    flash("Influencer deleted!", category="success")
    return redirect(url_for("admin.view_influencers"))
@admin.route("/view_flagged_influencers")
@role_required("Admin")
@login_required
def view_flagged_influencers():
    influencers = Influencer.query.filter_by(flagged=True).all()
    return render_template(
        "Admin/view_flagged_influencers.html", influencers=influencers
    )
```

```
@admin.route("/view_sponsors", methods=["GET", "POST"])
@role_required("Admin")
@login_required
def view_sponsors():
    if request.method == "POST":
        search_query = request.form.get("search_query")
        base_query = User.query.filter_by(role="Sponsor")
        sponsors = base_query.filter(User.name.ilike(f"%{search_query}%")).all()
    else:
        sponsors = User.query.filter_by(role="Sponsor").all()
    print(sponsors)
    return render_template("Admin/view_sponsors.html", sponsors=sponsors)

@admin.route("/view_flagged_sponsors", methods=["GET", "POST"])
@role_required("Admin")
@login_required
def view_flagged_sponsors():
    sponsors = User.query.filter_by(flagged=True).all()
    return render_template("Admin/view_flagged_sponsors.html", sponsors=sponsors)

@admin.route("flag_sponsor/<int:user_id>", methods=["POST"])
@role_required("Admin")
@login_required
def flag_sponsor(user_id):
    user = User.query.get_or_404(user_id)
    user.flagged = True
    db.session.commit()
    flash("Sponsor flagged!", category="success")
    return redirect(url_for("admin.view_sponsors"))

@admin.route("unflag_sponsor/<int:user_id>", methods=["POST"])
@role_required("Admin")
@login_required
def unflag_sponsor(user_id):
    user = User.query.get_or_404(user_id)
    user.flagged = False
    db.session.commit()
    flash("Sponsor reinstated!", category="success")
```

```
    return redirect(url_for("admin.view_flagged_sponsors"))
@admin.route("delete_sponsor/<int:user_id>", methods=["POST"])
@role_required("Admin")
@login_required
def delete_sponsor(user_id):
    user = User.query.get_or_404(user_id)
    for campaign in user.campaigns:
        if campaign:
            db.session.delete(campaign)
    for rating in user.ratings:
        if rating:
            db.session.delete(rating)
    for transaction in user.transactions:
        if transaction:
            transaction.user_id = None
    db.session.delete(user)
    db.session.commit()
    flash("Sponsor deleted!", category="success")
    return redirect(url_for("admin.view_sponsors"))
# Fetch transactions and group by date
@admin.route("/transaction_stats")
@role_required("Admin")
@login_required
def transaction_stats():
    transactions = (
        db.session.query(
            extract("year", Transaction.date).label("year"),
            extract("month", Transaction.date).label("month"),
            func.count(Transaction.id).label("count"),
        )
        .group_by(extract("year", Transaction.date), extract("month", Transaction.date))
        .all()
    )
    month_labels = [
        "Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
```



```

]
labels = []
transaction_counts = []
for record in transactions:
    year = int(record.year)
    month = int(record.month)
    labels.append(f'{month_labels[month - 1]} {year}')
    transaction_counts.append(record.count)
transaction_count = Transaction.query.count()
total_transaction_amount = (
    db.session.query(db.func.sum(Transaction.amount)).scalar() or 0
)
average_transaction_amount = (
    db.session.query(db.func.avg(Transaction.amount)).scalar() or 0
)
return render_template(
    "Admin/transactions_stats.html",
    labels=labels,
    transaction_counts=transaction_counts,
    transaction_count=transaction_count,
    total_transaction_amount=total_transaction_amount,
    average_transaction_amount=average_transaction_amount,
)

```

- **influencer.py**

```

from flask import (
    Blueprint,render_template,request,flash,redirect,url_for,current_app,abort,
)
from .models import User
from .models import (
    Influencer,AdRequest,Campaign,campaignRequest,Bookmark,Transaction,Rating,
    Negotiation,
)
from . import db
from flask_login import login_required, current_user

```

```
from werkzeug.utils import secure_filename
import os
from .views import role_required
from sqlalchemy.sql import func
from sqlalchemy import desc
influencer = Blueprint("influencer", __name__)
# -----INFLUENCER CRUD-----#
@influencer.route("/addInfluencer", methods=["GET", "POST"])
@role_required("Influencer")
@login_required
def addInfluencer():
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        profile_picture = request.files["profile_picture"]
        niche = request.form.get("niche")
        reach = request.form.get("reach")
        user_id = request.form["user_id"]
        platform = request.form.get("platform")
        if profile_picture:
            filename = secure_filename(profile_picture.filename)
            upload_folder = current_app.config["UPLOAD_FOLDER"]
            if not os.path.exists(upload_folder):
                os.makedirs(upload_folder)
            profile_picture.save(os.path.join(upload_folder, filename))
            profile_picture_path = os.path.join(upload_folder, filename)
            new_influencer = Influencer(
                name=name,
                email=email,
                profile_picture=filename,
                niche=niche,
                platform=platform,
                reach=reach,
                user_id=user_id,
            )
```

```
db.session.add(new_influencer)
db.session.commit()
flash("Influencer added successfully!", category="success")
return redirect(url_for("influencer.dashboard"))

else:
    flash("Profile picture is required!", "error")
    return render_template("Influencer/addInfluencer.html")
@influencer.route("/editInfluencer/<int:influencer_id>", methods=["GET", "POST"])
@role_required("Influencer")
@login_required
def editInfluencer(influencer_id):
    influencer = Influencer.query.get_or_404(influencer_id)
    if request.method == "POST":
        influencer.name = request.form["name"]
        influencer.email = request.form["email"]
        influencer.niche = request.form.get("niche")
        influencer.reach = request.form.get("reach")
        influencer.platform = request.form.get("platform")
        profile_picture = request.files["profile_picture"]
        if profile_picture:
            try:
                filename = secure_filename(profile_picture.filename)
                save_path = os.path.join(current_app.config["UPLOAD_FOLDER"], filename)
                profile_picture.save(save_path)
                influencer.profile_picture = filename
                db.session.commit()
                flash("Influencer updated successfully!", category="success")
                return redirect(url_for("influencer.dashboard"))
            except Exception as e:
                flash(f"Failed to save profile picture: {str(e)}", category="error")
    return render_template("Influencer/editInfluencer.html", influencer=influencer)
@influencer.route("/deleteInfluencer/<int:influencer_id>", methods=["POST"])
@role_required("Influencer")
@login_required
```

```
def deleteInfluencer(influencer_id):
    influencer = Influencer.query.get_or_404(influencer_id)
    if influencer.user_id != current_user.id:
        flash("You are not authorized to delete this profile.", "error")
        return redirect(url_for("influencer.viewInfluencers"))
    for rating in influencer.ratings:
        db.session.delete(rating)
    for transaction in influencer.transactions:
        transaction.influencer_id = None
    db.session.delete(influencer)
    db.session.commit()
    flash("Profile deleted successfully!", category="success")
    return redirect(url_for("influencer.dashboard"))
@influencer.route("/dashboard", methods=["GET", "POST"])
@role_required("Influencer")
@login_required
def dashboard():
    influencers = current_user.influencers
    influencer = influencers[0] if influencers else None
    if influencer:
        avg_rating_query = (
            db.session.query(func.avg(Rating.rating).label("average_rating"))
            .filter(Rating.ratee_id == influencer.id)
            .first()
        )
        average_rating = avg_rating_query.average_rating
    if influencer:
        ad_requests = AdRequest.query.filter_by(influencer_id=influencer.id).all()
        campaignRequests = campaignRequest.query.filter_by(
            influencer_id=influencer.id
        ).all()
        campaign_requests_with_details = []
        for campaign_request in campaignRequests:
            campaign = Campaign.query.get(campaign_request.campaign_id)
            if campaign:
```

```
        campaign_name = campaign.name
        campaign_requests_with_details.append(
            {
                "campaign_request": campaign_request,
                "campaign_name": campaign_name,
            }
        )
    else:
        flash("Campaign not found", category="error")
    ad_requests_with_details = []
    for ad_request in ad_requests:
        campaign = Campaign.query.get(ad_request.campaign_id)
        user = User.query.get(campaign.user_id)
        ad_requests_with_details.append(
            {
                "ad_request": ad_request,
                "campaign_name": campaign.name,
                "user_name": user.name,
            }
        )
    transactions = Transaction.query.filter_by(
        influencer_id=influencer.id
    ).all()
    # Calculate total earnings
    total_earnings = sum(transaction.amount for transaction in transactions)
    return render_template(
        "Influencer/dashboard.html",
        campaign_requests=campaign_requests_with_details,
        user=current_user,
        influencer=influencer,
        ad_requests=ad_requests_with_details,
        total_earnings=total_earnings,
        average_rating=average_rating,
    )
return render_template("Influencer/dashboard.html")
```

```
# -----Campaigns-----#
@influencer.route("/activeCampaigns", methods=["GET"])
@role_required("Influencer")
@login_required
def activeCampaigns():
    influencers = current_user.influencers
    influencer = influencers[0] if influencers else None
    ad_requests = AdRequest.query.filter_by(
        influencer_id=influencer.id, completed=False
    ).all()
    ad_requests_with_details = []
    for ad_request in ad_requests:
        campaign = Campaign.query.get(ad_request.campaign_id)
        user = User.query.get(campaign.user_id)
        ad_requests_with_details.append(
            {
                "request": ad_request,
                "campaign_name": campaign.name,
                "user_name": user.name,
                "request_type": "Received",
            }
        )
    campaign_requests = campaignRequest.query.filter_by(
        influencer_id=influencer.id, completed=False
    ).all()
    campaign_requests_with_details = []
    for campaign_request in campaign_requests:
        campaign = Campaign.query.get(campaign_request.campaign_id)
        user = User.query.get(campaign.user_id)
        campaign_requests_with_details.append(
            {
                "request": campaign_request,
                "campaign_name": campaign.name,
                "user_name": user.name,
                "request_type": "Sent",
```

```

    }
)
return render_template(
    "Influencer/activeCampaigns.html",
    user=current_user,
    influencer=influencer,
    ad_requests=ad_requests_with_details,
    campaign_requests=campaign_requests_with_details,
)
@influencer.route("/viewCampaigns", methods=["GET", "POST"])
@role_required("Influencer")
@login_required
def viewCampaigns():
    influencers = current_user.influencers
    influencer = influencers[0] if influencers else None
    if request.method == "POST":
        search_query = request.form.get("search_query")
        campaigns = Campaign.query.filter(
            Campaign.name.ilike(f"%{search_query}%")
        ).all()
    else:
        campaigns = Campaign.query.filter_by(flagged=False).all()
    for campaign in campaigns:
        campaign.is_bookmarked = (
            Bookmark.query.filter_by(
                influencer_id=current_user.id, campaign_id=campaign.id
            ).first()
            is not None
        )
    return render_template(
        "Influencer/viewCampaigns.html", campaigns=campaigns, influencer=influencer
    )
@influencer.route("/viewCampaign/<int:campaign_id>")
@role_required("Influencer")
@login_required

```

```
def viewCampaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)
    return render_template("Influencer/viewCampaign.html", campaign=campaign)
@influencer.route("/search_campaigns", methods=["GET", "POST"])
@login_required
@role_required("Influencer")
def search_campaigns():
    if request.method == "POST":
        search_query = request.form.get("search_query")
        campaigns = Campaign.query.filter(
            Campaign.name.ilike(f"%{search_query}%")
        ).all()
        for campaign in campaigns:
            campaign.is_bookmarked = (
                Bookmark.query.filter_by(
                    influencer_id=current_user.id, campaign_id=campaign.id
                ).first()
                is not None
            )
        return render_template(
            "Influencer/searchResults.html",
            campaigns=campaigns,
            search_query=search_query,
        )
    return render_template("Influencer/viewCampaigns.html")
@influencer.route(
    "/activeCampaigns/<int:ad_request_id>/mark_completed", methods=["POST"]
)
@login_required
@role_required("Influencer")
def mark_completed(ad_request_id):
    request_type = request.form.get("request_type")
    if request_type == "Received":
        ad_request = AdRequest.query.get_or_404(ad_request_id)
        if ad_request:
```



```

        ad_request.completed = True
        db.session.commit()
        flash("Ad request completed successfully.", category="success")
    if request_type == "Sent":
        campaign_request = campaignRequest.query.get_or_404(ad_request_id)
        if campaign_request:
            campaign_request.completed = True
            db.session.commit()
            flash("Ad request completed successfully.", category="success")
    return redirect(url_for("influencer.dashboard"))
# -----Ad Requests-----#
@influencer.route("/view_completed_requests", methods=["GET"])
@role_required("Influencer")
@login_required
def view_completed_requests():
    influencers = current_user.influencers
    influencer = influencers[0] if influencers else None
    ad_requests = AdRequest.query.filter_by(
        influencer_id=influencer.id, completed=True
    ).all()
    print(ad_requests)
    ad_requests_with_details = []
    for ad_request in ad_requests:
        campaign = Campaign.query.get(ad_request.campaign_id)
        user = User.query.get(campaign.user_id)
        ad_requests_with_details.append(
            {
                "request": ad_request,
                "campaign_name": campaign.name,
                "user_name": user.name,
                "request_type": "Received",
            }
        )
    campaign_requests = campaignRequest.query.filter_by(
        influencer_id=influencer.id, completed=True

```

```

).all()
campaign_requests_with_details = []
for campaign_request in campaign_requests:
    campaign = Campaign.query.get(campaign_request.campaign_id)
    user = User.query.get(campaign.user_id)
    campaign_requests_with_details.append(
        {
            "request": campaign_request,
            "campaign_name": campaign.name,
            "user_name": user.name,
            "request_type": "Sent",
        }
    )
return render_template(
    "Influencer/view_completed_requests.html",
    user=current_user,
    influencer=influencer,
    ad_requests=ad_requests_with_details,
    campaign_requests=campaign_requests_with_details,
)
@influencer.route("/viewRequest/<int:ad_request_id>", methods=["GET"])
@role_required("Influencer")
@login_required
def viewRequest(ad_request_id):
    ad_request = AdRequest.query.get_or_404(ad_request_id)
    # Retrieve the currently logged-in influencer
    influencer = Influencer.query.filter_by(user_id=current_user.id).first()
    # Check if the ad request belongs to the currently logged-in influencer
    if ad_request.influencer_id != influencer.id:
        abort(403)
    campaign = Campaign.query.get(ad_request.campaign_id)
    user = User.query.get(campaign.user_id)
    ad_request_with_details = {
        "ad_request": ad_request,
        "campaign_name": campaign.name,

```

```
        "user_name": user.name,
    }
    negotiations = (
        Negotiation.query.filter_by(
            ad_request_id=ad_request.id, receiver_id=current_user.id
        )
        .order_by(desc(Negotiation.created_at))
        .all()
    )
    latest_nego = (
        Negotiation.query.filter_by(
            ad_request_id=ad_request.id, receiver_id=current_user.id, status="Accepted"
        )
        .order_by(desc(Negotiation.created_at))
        .first()
    )
    latest_pending_nego = (
        Negotiation.query.filter_by(
            ad_request_id=ad_request.id, receiver_id=current_user.id, status="Pending"
        )
        .order_by(desc(Negotiation.created_at))
        .first()
    )
    print(latest_pending_nego)
    latest_price = (
        latest_nego.offer_amount if latest_nego else ad_request.payment_amount
    )
    return render_template(
        "Influencer/viewRequest.html",
        ad_request_details=ad_request_with_details,
        negotiations=negotiations,
        latest_price=latest_price,
        latest_pending_nego=latest_pending_nego,
    )
    @influencer.route(
```

```
"/edit_campaign_request/<int:campaign_request_id>", methods=["GET", "POST"]
)
@role_required("Influencer")
@login_required
def edit_campaign_request(campaign_request_id):
    campaign_request = campaignRequest.query.get_or_404(campaign_request_id)
    if request.method == "POST":
        campaign_request.messages = request.form.get("messages")
        campaign_request.requirements = request.form.get("requirements")
        campaign_request.payment_amount = request.form.get("payment_amount")
        db.session.commit()
        flash("Campaign request updated successfully.", category="success")
        return redirect(url_for("influencer.dashboard"))
    return render_template(
        "Influencer/edit_sent_request.html", campaign_request=campaign_request
    )
@influencer.route(
    "/delete_campaign_request/<int:campaign_request_id>", methods=["POST"]
)
@role_required("Influencer")
@login_required
def delete_campaign_request(campaign_request_id):
    campaign_request = campaignRequest.query.get_or_404(campaign_request_id)
    db.session.delete(campaign_request)
    db.session.commit()
    flash("Campaign request deleted successfully.", category="success")
    return redirect(url_for("influencer.dashboard"))
@influencer.route("/ad_request/<int:ad_request_id>/accept", methods=["POST"])
@role_required("Influencer")
@login_required
def accept_ad_request(ad_request_id):
    ad_request = AdRequest.query.get_or_404(ad_request_id)
    payment_amount = request.form.get("payment_amount")
    ad_request.status = "Accepted"
    ad_request.payment_amount = payment_amount
```

```
db.session.commit()

flash("Ad request accepted successfully.", category="success")

return redirect(url_for("influencer.dashboard"))

@influencer.route("/ad_request/<int:ad_request_id>/reject", methods=["POST"])
@role_required("Influencer")
@login_required
def reject_ad_request(ad_request_id):
    ad_request = AdRequest.query.get_or_404(ad_request_id)
    ad_request.status = "Rejected"
    db.session.commit()
    flash("Ad request rejected successfully.", category="success")
    return redirect(url_for("influencer.dashboard"))

@influencer.route("create_ad_request/<int:campaign_id>", methods=["GET", "POST"])
@role_required("Influencer")
@login_required
def create_ad_request(campaign_id):
    campaign = Campaign.query.get(campaign_id)
    influencer = Influencer.query.filter_by(user_id=current_user.id).first()
    if not campaign:
        flash("Campaign not found", "error")
        return redirect(url_for("sponsor.dashboard"))
    if request.method == "POST":
        messages = request.form.get("messages")
        requirements = request.form.get("requirements")
        payment_amount = request.form.get("payment_amount")
        if not influencer:
            flash("Influencer name is required", "error")
            return render_template(
                "Influencer/create_ad_request.html", campaign=campaign
            )
        ad_request = AdRequest(
            campaign_id=campaign.id,
            influencer_id=influencer.id,
            messages=messages,
            requirements=requirements,
```

```

        payment_amount=payment_amount,
    )
    db.session.add(ad_request)
    db.session.commit()
    flash("Ad request created successfully", category="success")
    return redirect(url_for("influencer.dashboard"))
    return render_template("Influencer/create_ad_request.html", campaign=campaign)
# -----Bookmark Campaign-----#
@influencer.route("/bookmark_campaign/<int:campaign_id>", methods=["POST"])
@login_required
@role_required("Influencer")
def bookmark_campaign(campaign_id):
    if request.method == "POST":
        # Check if the campaign is already bookmarked
        bookmark = Bookmark.query.filter_by(
            influencer_id=current_user.id, campaign_id=campaign_id
        ).first()
        if bookmark:
            # If already bookmarked, remove the bookmark
            db.session.delete(bookmark)
            db.session.commit()
        else:
            # If not bookmarked, create a new bookmark entry
            new_bookmark = Bookmark(
                influencer_id=current_user.id, campaign_id=campaign_id
            )
            db.session.add(new_bookmark)
            db.session.commit()
        return redirect(url_for("influencer.viewCampaigns", campaign_id=campaign_id))
    @influencer.route("/view_bookmarks")
    @login_required
    @role_required("Influencer")
    def view_bookmarks():
        influencers = current_user.influencers
        influencer = influencers[0] if influencers else None

```

```

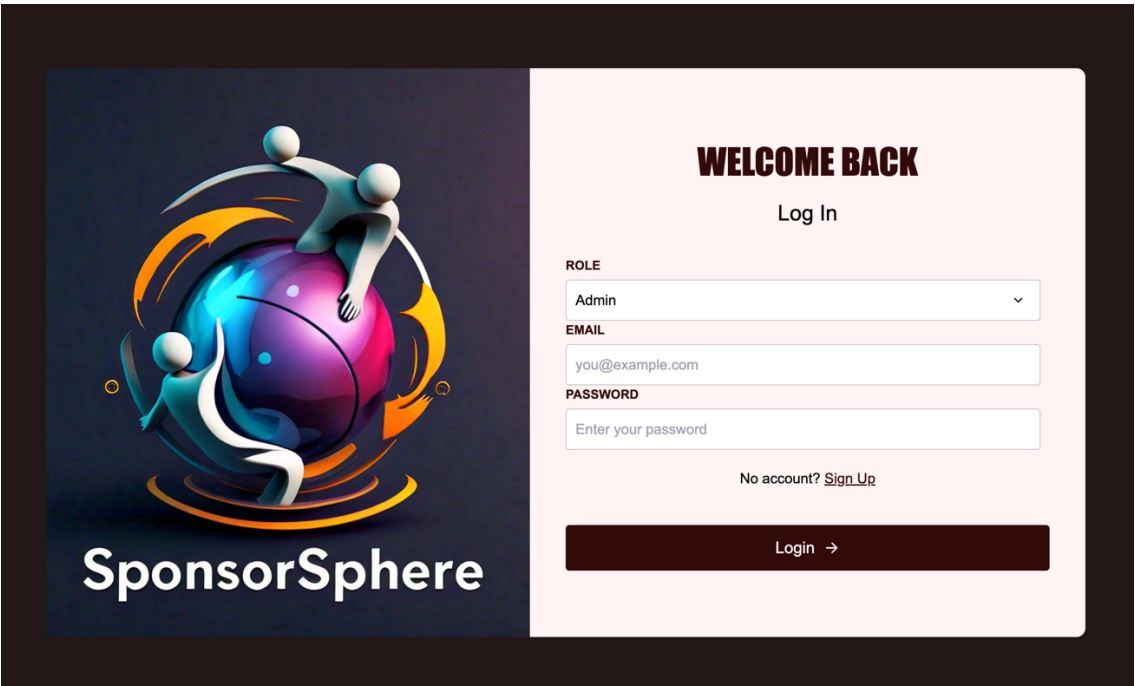
campaigns = Campaign.query.all()
for campaign in campaigns:
    campaign.is_bookmarked = (
        Bookmark.query.filter_by(
            influencer_id=current_user.id, campaign_id=campaign.id
        ).first()
        is not None
    )
influencer_bookmarks = Bookmark.query.filter_by(influencer_id=current_user.id).all()
bookmarked_campaigns = [
    Campaign.query.get(bookmark.campaign_id) for bookmark in influencer_bookmarks
]
return render_template(
    "Influencer/Bookmarks.html",
    bookmarked_campaigns=bookmarked_campaigns,
    campaigns=campaigns,
    influencer=influencer,
)
# -----Ratings-----#
@influencer.route("/view_ratings")
@role_required("Influencer")
@login_required
def view_ratings():
    influencers = current_user.influencers
    influencer = influencers[0] if influencers else None
    ratings = Rating.query.filter_by(ratee_id=influencer.id).all()
    ratings_with_details = []
    for rating in ratings:
        sponsor = User.query.get(rating.rater_id)
        sponsor_name = sponsor.name
        ratings_with_details.append(
            {
                "rating": rating,
                "sponsor_name": sponsor_name,
            }
        )

```

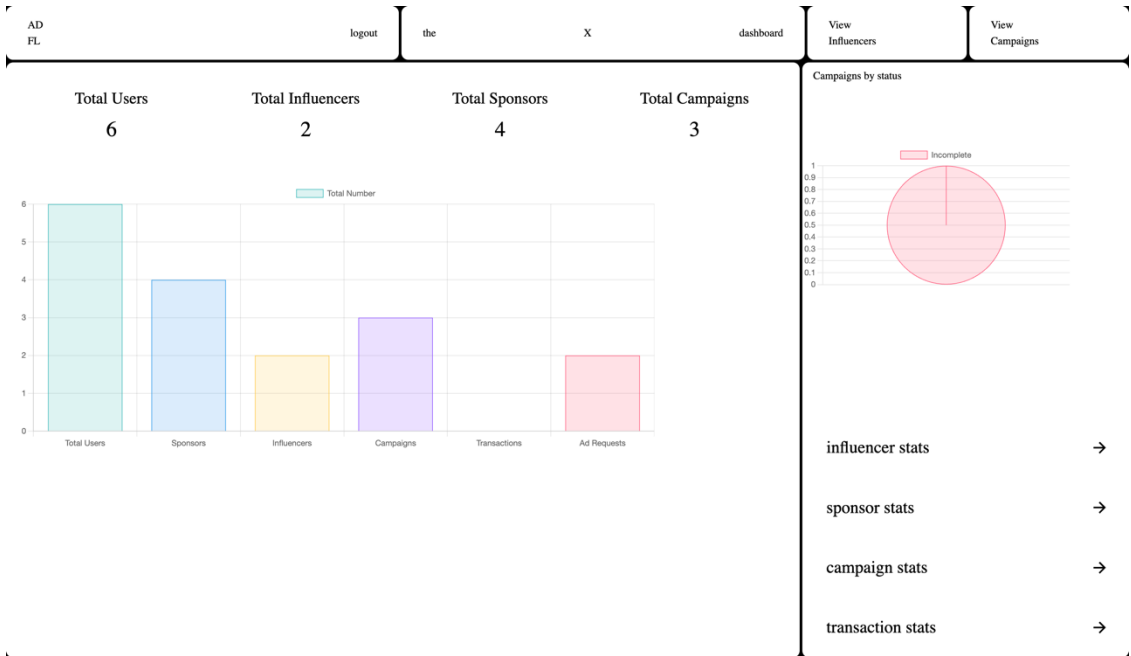
```
)
    return render_template("Influencer/view_ratings.html", ratings=ratings_with_details)
@influencer.route("make_ad_offer/<int:ad_request_id>", methods=["POST", "GET"])
@role_required("Influencer")
@login_required
def make_ad_offer(ad_request_id):
    ad_request = AdRequest.query.get_or_404(ad_request_id)
    campaign = Campaign.query.get(ad_request.campaign_id)
    sponsor = User.query.get(campaign.user_id)
    sender_id = sponsor.id
    if request.method == "POST":
        offer_amount = request.form.get("offer_amount")
        negotiation = Negotiation(
            campaign_id=campaign.id,
            ad_request_id=ad_request_id,
            offer_amount=offer_amount,
            sender_id=sender_id,
            receiver_id=current_user.id,
        )
        db.session.add(negotiation)
        db.session.commit()
        flash("Offer created successfully", "success")
        return redirect(url_for("influencer.dashboard"))
    return render_template("Influencer/dashboard.html", ad_request=ad_request)
```


10. Screenshots

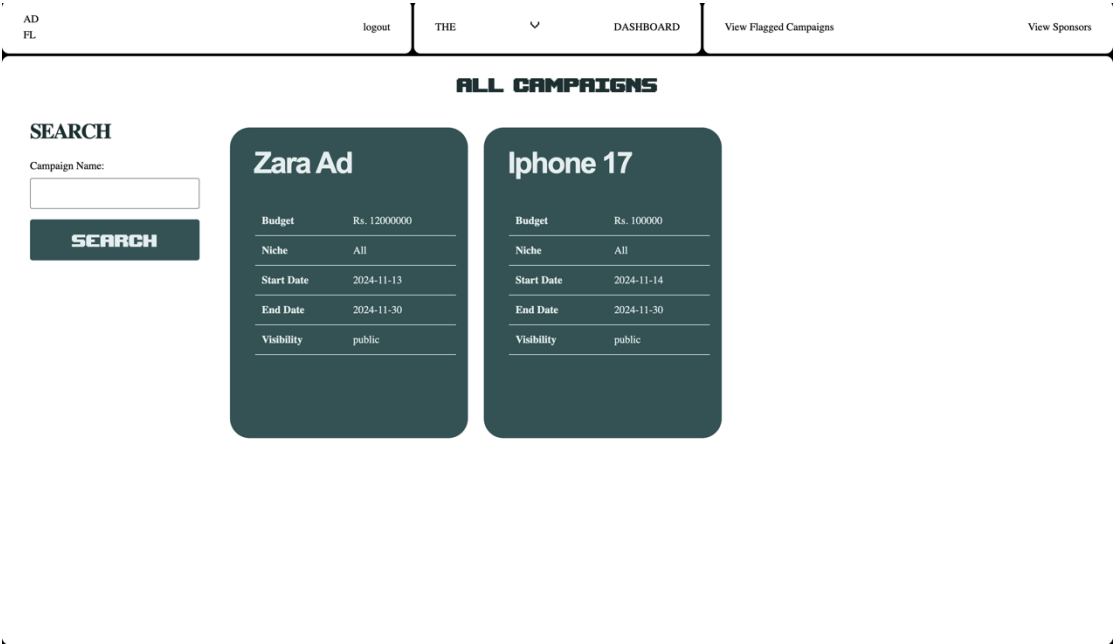
- Login page



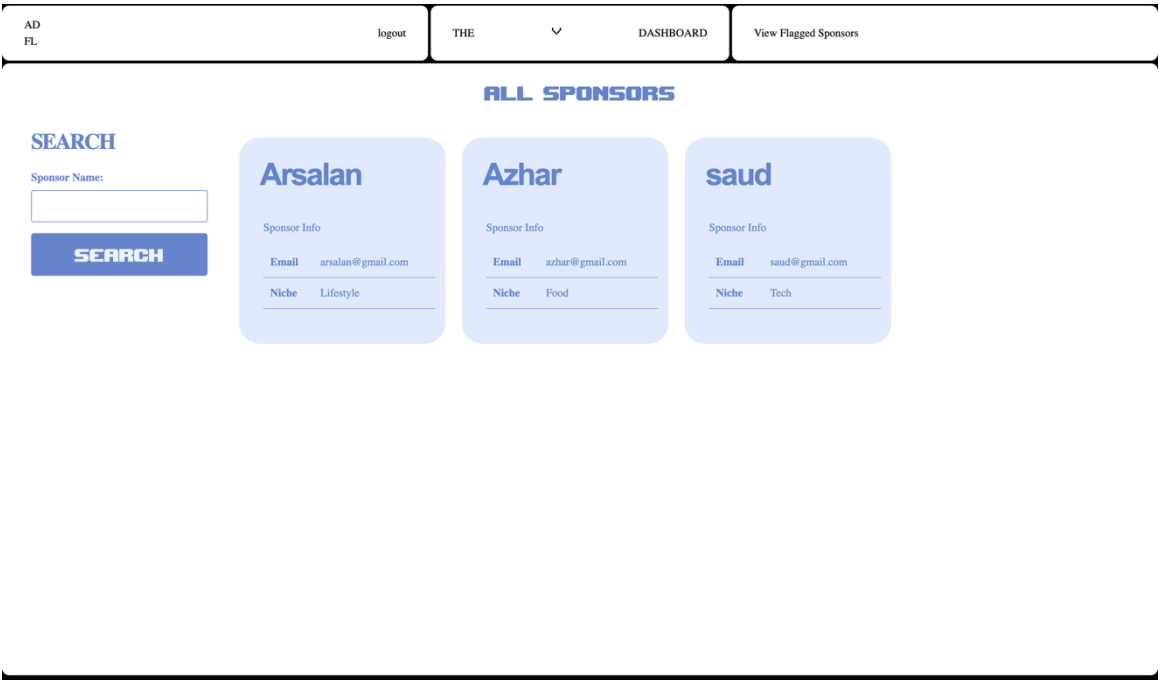
- AdminDashboard



- AdminViewCampaigns



- AdminViewSponsors



- SponsorsDashboard

AD
FL

logoutTHEH

DASHBOARD

Ratings

view
campaigns

INFLUENCER REQUESTS

SNO.	Campaign	Influence	Task	Message	Asked Amount
------	----------	-----------	------	---------	-----------------

Sent Requests

SNO.	Campaign	Influencer	Status	Payment Amount Rs.		
1	Iphone 17	saif	Pending	120000.0	Delete	View
2	Zara Ad	saif	Accepted	12380.0	Delete	View

- SponsorsViewCampaigns

AD
FL

logoutTHE

DASHBOARD

find
Influencers

completed
requests

CAMPAIGN

XOXOXOXO

Campaign Detail	Information
Name	Zara Ad
Description	create a vlog
Start Date	2024-11-13
End Date	2024-11-30
Budget	12000000
Visibility	public
Goals	get 1000 buyers
Status	Incomplete
Mark as Completed	Yes

AD REQUESTS

Request Type	Influencer	Messages	Requirements	Amount	Status	Completion Status
Sent	saif	accept my proposal	i will complete graphics	12380.0	Accepted	False

• InfluencersDashboard

AD
FL

logout

THE X DASHBOARD

Find campaigns

active campaigns

Monis's profile picture

Monis

Niche

Reach

Rating

Lifestyle

100000

None

Pending Requests

Campaign

Sponsor

Payment Amount

Sent Requests

Campaign

Requirements

Message

Requested Amount

• InfluencersSearchCampaigns

ALL CAMPAIGNS

SEARCH

Campaign Name:

SEARCH

ZARA AD

Budget

Rs. 12000000

Niche

All

Start Date

2024-11-13

End Date

2024-11-30

Visibility

public

IPHONE 17

Budget

Rs. 100000

Niche

All

Start Date

2024-11-14

End Date

2024-11-30

Visibility

public

APPENDIX : ISSUES LIST

1. **Real-Time Updates:** We need to assess the feasibility of implementing real-time tracking of sponsor-influencer interactions and campaign performance metrics. This is a high-priority item, as it directly impacts the core functionality and user experience of SponsorSphere.
2. **Data Privacy Policy:** Finalizing and documenting privacy policies for handling user data (sponsors and influencers) is essential to ensure compliance with regulations such as GDPR and CCPA. This issue is marked TBD and is a high priority.
3. **Localization Requirements:** A decision needs to be made regarding which languages should be supported in the initial release to cater to a diverse user base. This is a medium-priority issue, as it impacts usability and accessibility for international users.
4. **User Feedback Mechanism:** We must decide whether to implement an in-app feedback tool that allows sponsors and influencers to provide feedback or suggest improvements. This is a medium-priority item, but it could significantly enhance user engagement and satisfaction. The development team will evaluate feasibility and implementation requirements.
5. **Backup Frequency:** The frequency and retention policy for database backups need to be established to meet reliability requirements and ensure data recovery capabilities. This is a high-priority item that remains TBD.
6. **Scalability Considerations:** An analysis is needed to ensure that the system can handle increasing numbers of users and campaigns as SponsorSphere grows. This is a medium-priority issue, as it will affect long-term performance and reliability.
7. **Integration with Payment Gateways:** We need to evaluate the integration of payment gateways for handling transactions between sponsors and influencers. This is a high priority item, given its importance for revenue generation and user trust.
8. **Compliance with Advertising Regulations:** It's necessary to ensure that the platform complies with advertising regulations and standards relevant to influencer marketing. This issue is marked TBD and is of high priority to avoid legal complications.
9. **Performance Monitoring Tools:** Deciding on performance monitoring tools to track system performance and user engagement is crucial for maintaining optimal operation. This is a medium-priority issue that requires further exploration.